Neural Network Linear Algebra

EEC 174AY Applied Machine Learning

Ahmed Khan

University of California, Davis

October 2, 2026

Contents



Vectors, Matrices, and Tensors

- Vectors
- The Dual Space
- Linear Combinations
- Standard Basis Vectors
- Rules for Vector Operations
- Matrices
- Linear Transformations
- Norms
- Matrix Addition and Scalar Multiplication
- The Hadamard Product
- Element-wise Functions
- The Transpose
- Tensors
- Tensor Notation

Products

- The Inner Product
- The Outer Product
- Broadcasting
- Matrix-Vector Products
- Matrix Multiplication
- The Rows and Columns of a Matrix Product
- Composition of Linear Transformations
- Rules for Matrix Operations
- The Identity Matrix and Matrix Units
- Moving a Matrix Across an Inner Product

Contents (continued)



The Frobenius Inner Product

- The Frobenius Inner Product
- Lengths and Angles Between Matrices
- Linearity of the Frobenius Inner Product
- Linear Functions of Matrices
- The Trace and the Frobenius Inner Product
- Cyclic Permutations of the Trace
- Moving Factors Across the Frobenius Inner Product
- Tensor Products
- Polynomial Forms
- Multilinear Forms
- Tensor Contraction
- Summation Notation
- Operations in Summation Notation

Differentials

- Partial Derivatives
- The Differential Is a Linear Functional
- The Gradient Vector
- How a Gradient Acts on a Differential
- Gradients of Linear and Quadratic Functions
- Matrix Gradients
- Rules for Differentials
- Differentials of Element-wise Functions
- Rules of Matrix Differentials
- Reading Off a Gradient
- Reading Off a Gradient Through a Product

Contents (continued)



The Forward Pass

- The Network as Neurons and as a Function
- The Network for a Batch
- Vectorization
- Data Matrices
- Activation Functions
- Linear Separability
- Logits
- The Softmax Function
- Vectorizing the Softmax
- Cross-Entropy Loss
- Vectorizing the Cross-Entropy Loss
- Forward Propagation
- From Forward Propagation to a Graph



The Backward Pass

Going Backward
The Chain Rule Along Each Path
Splitting the Graph at P
The Differential of the Log-Probabilities
The Long Way: The Loss's Gradient
The Long Way: The Softmax's Gradient
The Long Way: The Softmax's Gradient, Continued
The Long Way: Combining the Two
Splitting the Graph at ln P
The Two Splits in PyTorch
The Gradient at the Logits
Backward Steps 1 and 2 in Full
Backward Step 3: The Second Affine Map
The Differential of the Second Affine Map
Gradients of the Second Affine Map
Backward Step 3 in Full
Backward Step 4: The Sigmoid
Gradients Through the Sigmoid
Backward Step 4 in Full
Backward Step 5: The First Affine Map
Gradients of the First Affine Map
Backward Step 5 in Full
Putting It Together: Passing Gradients Back
Putting It Together: The Parameter Gradients
The Backward Pass in PyTorch
L2 Regularization
Weight Decay
Reverse-Mode Automatic Differentiation
The Forward and Backward Passes

Vectors, Matrices, and Tensors

Vectors

A vector is an ordered list of scalars that we can also draw as an arrow from the origin.

$$\mathbf{x} = \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix} \in \mathbb{R}^3$$

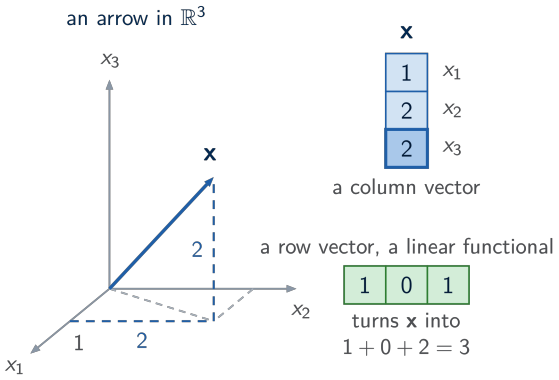
We write vectors as columns.

$$\begin{bmatrix} 1 & 0 & 1 \end{bmatrix} \in \mathbb{R}^{1 \times 3}$$

A row vector is an ordered list of scalars written across. Row vectors add and scale like column vectors, so they form a vector space of their own called the dual space.

$$\mathbf{x} \in \mathbb{R}^{784}$$

Dimensions higher than 3 cannot be visualized, but they operate in the same way.



The Dual Space

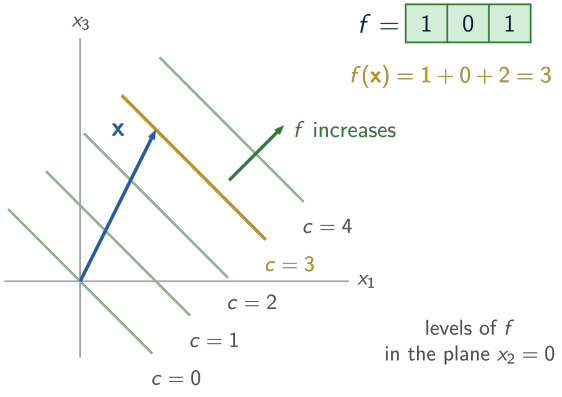
Row vectors form the dual space: the linear functionals that turn column vectors into scalars.

$f(\mathbf{x}) = a_1x_1 + a_2x_2 + a_3x_3$

Every linear functional on $\mathbb{R}^3$ has this form, so it is described completely by the row vector $\begin{bmatrix} a_1 & a_2 & a_3 \end{bmatrix}$.

$f(\mathbf{x}) = c$

Geometrically, a row vector is a family of parallel, evenly spaced planes, one for each value c; $f(\mathbf{x})$ is the value of the plane that $\mathbf{x}$ reaches.



Linear Combinations

Scaling vectors and adding them gives a linear combination.

$a\,\mathbf{u} + b\,\mathbf{v}$

Scaling multiplies each entry; adding adds matching entries.

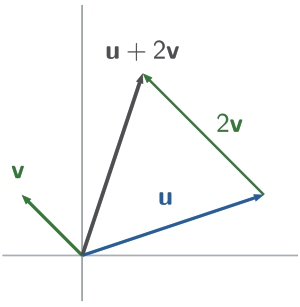
$c_1\mathbf{v}_1 + c_2\mathbf{v}_2 + \cdots + c_n\mathbf{v}_n$

Any number of vectors can be combined, with any weights.

$f(a\mathbf{u} + b\mathbf{v}) = a\,f(\mathbf{u}) + b\,f(\mathbf{v})$

Superposition: applying a linear function to a combination of vectors gives the same combination of its outputs.

drawn head to tail



computed element-wise

$$2 \cdot \begin{matrix} \mathbf{v} \\ \boxed{\begin{matrix} -1 \\ 1 \end{matrix}} \end{matrix} = \begin{matrix} 2\mathbf{v} \\ \boxed{\begin{matrix} -2 \\ 2 \end{matrix}} \end{matrix}$$

scale each entry

$$\begin{matrix} \mathbf{u} \\ \boxed{\begin{matrix} 3 \\ 1 \end{matrix}} \end{matrix} + \begin{matrix} 2\mathbf{v} \\ \boxed{\begin{matrix} -2 \\ 2 \end{matrix}} \end{matrix} = \begin{matrix} \mathbf{u} + 2\mathbf{v} \\ \boxed{\begin{matrix} 1 \\ 3 \end{matrix}} \end{matrix}$$

add matching entries

Standard Basis Vectors

Every vector is a linear combination of the standard basis vectors.

$$\mathbf{x} = x_1\mathbf{e}_1 + x_2\mathbf{e}_2 + x_3\mathbf{e}_3$$

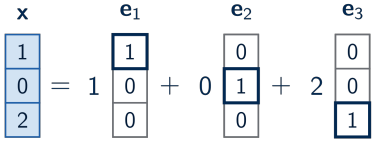
$\mathbf{e}_j$ has a 1 in position j and 0 elsewhere, which is called one-hot, and the entries of $\mathbf{x}$ are the weights.

$$\mathbf{1} = \mathbf{e}_1 + \mathbf{e}_2 + \mathbf{e}_3$$

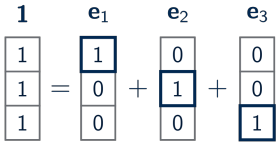
The vector of ones gives every basis vector the weight 1.

$$\mathbf{e}_j^\mathsf{T}\mathbf{x} = x_j, \quad \mathbf{1}^\mathsf{T}\mathbf{x} = \textstyle\sum_i x_i$$

Read as row vectors, $\mathbf{e}_j^\mathsf{T}$ picks out entry j and $\mathbf{1}^\mathsf{T}$ adds up all the entries.



the entries of $\mathbf{x}$ are the weights



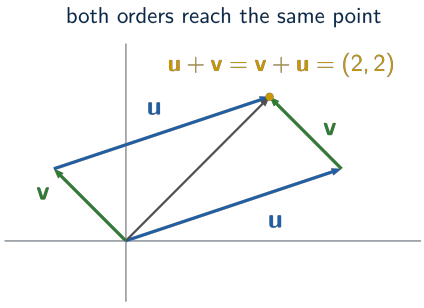
every weight is 1

Rules for Vector Operations

Vector addition and scalar multiplication follow the same rules as adding and multiplying numbers.

commutative	$\mathbf{u} + \mathbf{v} = \mathbf{v} + \mathbf{u}$
associative	$(\mathbf{u} + \mathbf{v}) + \mathbf{w} = \mathbf{u} + (\mathbf{v} + \mathbf{w})$
distributive over vectors	$a(\mathbf{u} + \mathbf{v}) = a\mathbf{u} + a\mathbf{v}$
distributive over scalars	$(a + b)\,\mathbf{u} = a\mathbf{u} + b\mathbf{u}$
zero vector	$\mathbf{u} + \mathbf{0} = \mathbf{u}$
negative	$\mathbf{u} + (-\mathbf{u}) = \mathbf{0}$

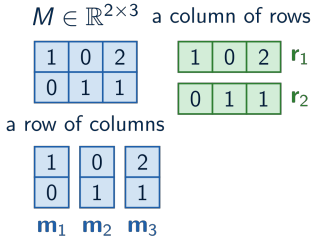
both orders reach the same point

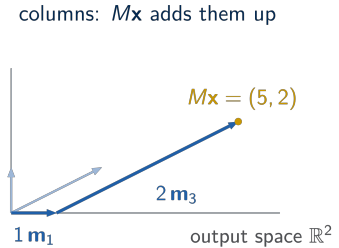


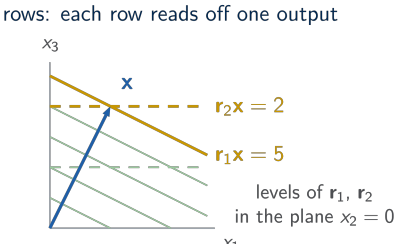
Each rule holds because it holds for every entry, so vectors can be added in any order and a scalar can be distributed over a sum.

Matrices

A matrix is a row of columns, a column of rows, and a linear transformation.







$$M\mathbf{x} = x_1\mathbf{m}_1 + x_2\mathbf{m}_2 + x_3\mathbf{m}_3$$

Columns live in the output space: $M\mathbf{x}$ adds them up, weighted by the entries of $\mathbf{x}$.

$$(M\mathbf{x})_i = \mathbf{r}_i\,\mathbf{x}$$

Rows live in the input space as linear functionals: output i is the level of row i that $\mathbf{x}$ reaches.

Linear Transformations

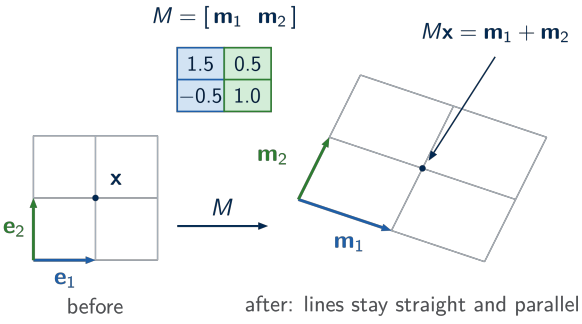
A linear transformation moves every point of space, and the columns of its matrix show where the axes go.

$M\mathbf{e}_j = \mathbf{m}_j$

The j-th axis direction $\mathbf{e}_j$ lands on column j. With a 2×2 matrix we can watch the whole plane move.

$M\mathbf{x} = x_1\mathbf{m}_1 + x_2\mathbf{m}_2$

Every other point lands at the same linear combination of the columns, so grid lines stay straight, parallel, and evenly spaced, and the origin stays fixed.



Norms

The norm of a vector is its length, and the same sum of squares measures the size of a matrix.

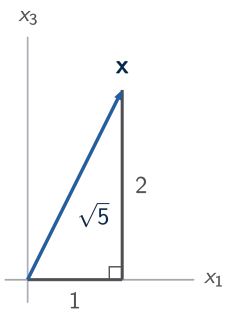
$$\|\mathbf{x}\| = \sqrt{x_1^2 + x_2^2 + x_3^2} = \sqrt{5}$$

This is the Pythagorean theorem: $\mathbf{x}$ goes 1 along the first axis and 2 up the third, so its length is $\sqrt{1+4}$.

$$\|M\|^2 = \sum_{i,j} M_{ij}^2 = 7$$

For a matrix, square every entry and add them all. This is the Frobenius norm.

the length of $\mathbf{x} = [1, 0, 2]^\mathsf{T}$



the size of M

M

1	0	2
0	1	1

square each entry

1	0	4
0	1	1

$1 + 0 + 4 + 0 + 1 + 1 = 7$

$\|M\|^2 = 7$

Matrix Addition and Scalar Multiplication

Matrices of the same shape add element-wise, and a scalar scales every entry.

$(U + V)_{ij} = U_{ij} + V_{ij}$

The two shapes must match.

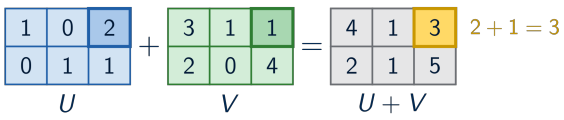
$(cU)_{ij} = c\,U_{ij}$

So matrices add and scale like vectors: they form a vector space of their own.

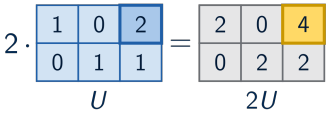
$(c - U)_{ij} = c - U_{ij}$

A scalar added to or subtracted from a matrix acts on every entry.

add matching entries



scale every entry



The Hadamard Product

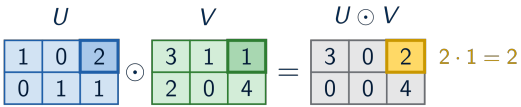
The Hadamard product multiplies matching entries, one scale factor per entry.

$(U \odot V)_{ij} = U_{ij} V_{ij}$

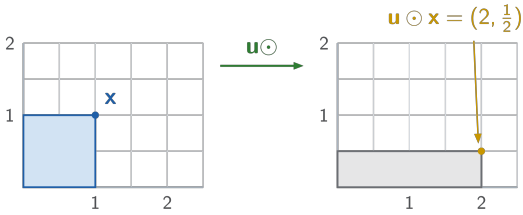
The two shapes must match.

u ⊙ x

For a constant vector **u**, this scales each axis by its own factor.



with a fixed $\mathbf{u} = (2, \frac{1}{2})$, each axis gets its own scale factor

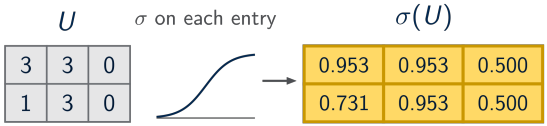


Element-wise Functions

An element-wise function applies one scalar function to every entry, so the shape does not change.

$$\sigma(U)_{ij} = \sigma(U_{ij}), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}$$

The sigmoid σ maps every entry into $(0, 1)$, large entries near 1 and very negative ones near 0. The exponential and the natural logarithm act on arrays the same way.



$$\sigma(U + V) \neq \sigma(U) + \sigma(V)$$

Unlike a matrix product, an element-wise function such as the sigmoid is not linear, so it bends straight lines instead of keeping them straight.

$$\sigma(1 + 1) = \sigma(2) = 0.881$$

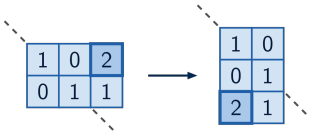
$$\sigma(1) + \sigma(1) = 0.731 + 0.731 = 1.462$$

they differ, so σ is not linear

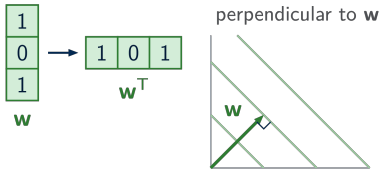
The Transpose

The transpose swaps two indices of an array: for a matrix, rows become columns.

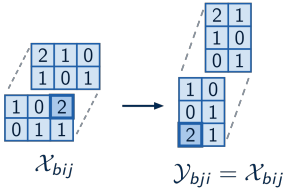
a matrix flips across its main diagonal



a column becomes a linear functional



a tensor swaps the indices it names



$(M^{\mathsf{T}})_{ij} = M_{ji}$

It flips across its main diagonal, and flipping twice gives back M.

$\mathbf{w}^{\mathsf{T}} = \begin{bmatrix} 1 & 0 & 1 \end{bmatrix}$

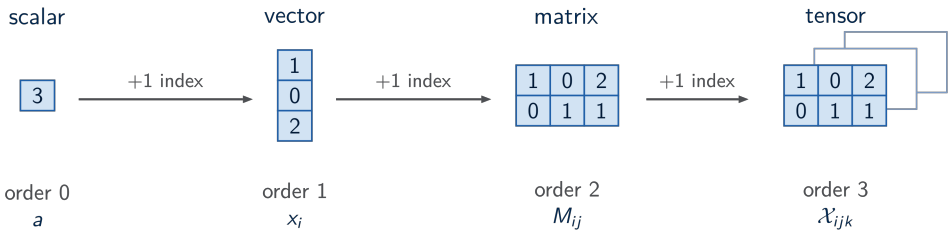
A column vector becomes a row vector, a linear functional whose levels are perpendicular to $\mathbf{w}$.

$\mathcal{Y}_{bji} = \mathcal{X}_{bij}$

A tensor's transpose names the two indices it swaps; here each slice b is transposed.

Tensors

Scalars, vectors, matrices, and tensors are arrays with 0, 1, 2, and 3 or more indices.



The number of indices is the order of the array. Each new index adds an axis, so a tensor is a stack of matrices.

Tensor Notation

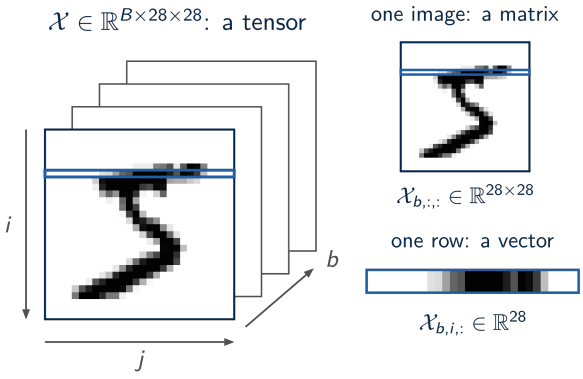
An index picks one position along each axis, and a colon keeps the whole axis.

$\mathcal{X} \in \mathbb{R}^{B\times 28\times 28}, \quad \mathcal{X}_{bij}$

Entry $\mathcal{X}_{bij}$ is pixel (i, j) of image b.

$\mathcal{X}_{b,:,:} \in \mathbb{R}^{28\times 28}, \quad \mathcal{X}_{b,i,:} \in \mathbb{R}^{28}$

Fixing an index removes its axis: one image is a matrix, and one row of pixels is a vector.



Products

The Inner Product

The inner product multiplies matching entries and adds them, and it measures how much two vectors point the same way.

$$\langle \mathbf{w}, \mathbf{x} \rangle = \mathbf{w}^\mathsf{T}\mathbf{x} = \sum_i w_i x_i$$

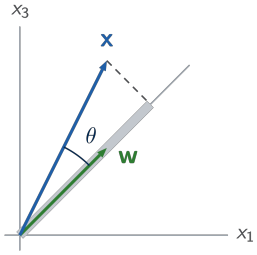
A row times a column gives a scalar, here $1 \cdot 1 + 0 \cdot 0 + 1 \cdot 2 = 3$.

$$\mathbf{w}^\mathsf{T}\mathbf{x} = \|\mathbf{w}\| \, \|\mathbf{x}\| \cos\theta$$

It is positive when the vectors point the same way, zero when they are perpendicular, and negative when they point apart. Divided by $\|\mathbf{w}\|$, it is the length of the projection of $\mathbf{x}$ onto $\mathbf{w}$.

$\mathbf{w}^\mathsf{T}$ | $\mathbf{x}$

1	0	1	×	1 0 2	=	3	$1 \cdot 1 + 0 \cdot 0 + 1 \cdot 2 = 3$



$$\mathbf{w}^\mathsf{T}\mathbf{x} = \|\mathbf{w}\| \, \|\mathbf{x}\| \cos\theta$$
$$3 = \sqrt{2} \cdot \sqrt{5} \cdot \cos\theta$$

the projection of $\mathbf{x}$ onto $\mathbf{w}$, in gray, has length $\mathbf{w}^\mathsf{T}\mathbf{x}/\|\mathbf{w}\| = 3/\sqrt{2}$

The Outer Product

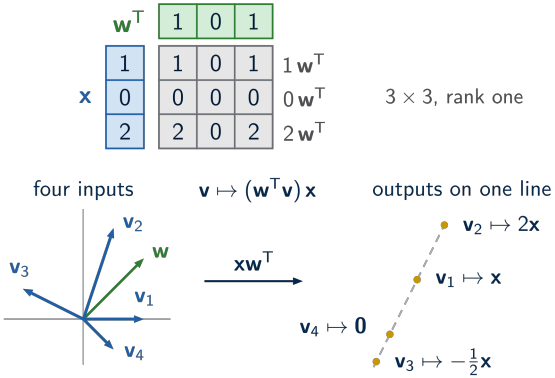
The outer product of two vectors is the matrix of all their pairwise products, and as a map it sends all of space onto one line.

$(\mathbf{x}\mathbf{w}^\mathsf{T})_{ij} = x_i w_j$

Every row is a multiple of $\mathbf{w}^\mathsf{T}$ and every column is a multiple of $\mathbf{x}$, so the matrix has rank one.

$(\mathbf{x}\mathbf{w}^\mathsf{T})\mathbf{v} = (\mathbf{w}^\mathsf{T}\mathbf{v})\,\mathbf{x}$

$\mathbf{w}^\mathsf{T}$ measures $\mathbf{v}$ as a scalar and $\mathbf{x}$ sets the direction, so every output is a multiple of $\mathbf{x}$; inputs perpendicular to $\mathbf{w}$ go to $\mathbf{0}$.



Broadcasting

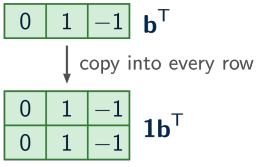
Adding $\mathbf{1}\mathbf{b}^\mathsf{T}$ copies the row $\mathbf{b}^\mathsf{T}$ into every row of a matrix, so every row moves by the same vector.

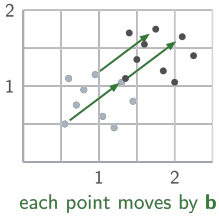
$U + \mathbf{1}\mathbf{b}^\mathsf{T}$

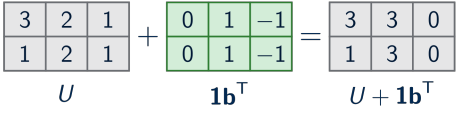
Here $\mathbf{1}$ is the vector of ones, one entry per row of U, and the outer product $\mathbf{1}\mathbf{b}^\mathsf{T}$ has $\mathbf{b}^\mathsf{T}$ in every row. Adding a row this way is called broadcasting.

$(\mathbf{1}\mathbf{b}^\mathsf{T})_{ij} = b_j$

The entry does not depend on the row i, so if each row of U is a point, every point is shifted by $\mathbf{b}$.







Matrix-Vector Products

A matrix times a vector is an inner product with each row, or a sum of outer products with the columns.

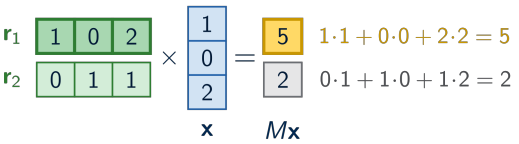
$$(M\mathbf{x})_i = \sum_k M_{ik} x_k$$

Entry i is the inner product of row i of M with $\mathbf{x}$.

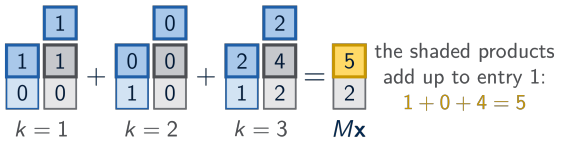
$$M\mathbf{x} = \sum_k M_{:,k}\, x_k$$

Row k of $\mathbf{x}$ is the single entry x_k, so column k of M times it is an outer product one column wide, and these add up to $M\mathbf{x}$.

rows: one inner product per row



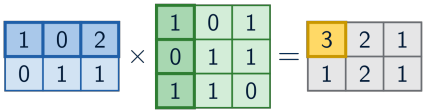
columns: one outer product per column



Matrix Multiplication

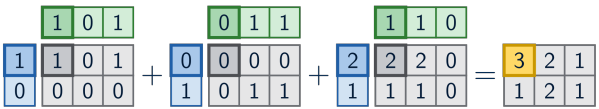
A matrix product keeps both views: inner products and outer products.

one entry is an inner product



$(MN)_{11} = 1\cdot 1 + 0\cdot 0 + 2\cdot 1 = 3$

the whole product is a sum of outer products



the shaded products add up to entry $(1,1)$: $1 + 0 + 2 = 3$

$$(MN)_{ij} = \sum_k M_{ik} N_{kj}$$

Entry (i, j) pairs row i of M with column j of N, so a $p \times q$ times a $q \times r$ is $p \times r$.

$$MN = \sum_k M_{:,k}\, N_{k,:}$$

Row k of N now has three entries, so each outer product is a full layer.

The Rows and Columns of a Matrix Product

Each column of MN is M times a column of N; each row is a row of M times N.

by columns: M transforms each column of N

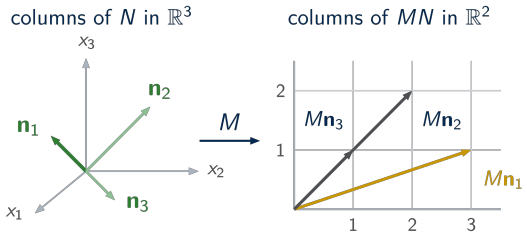
$$\begin{bmatrix}1&0&2\\0&1&1\end{bmatrix} \times \begin{bmatrix}1&0&1\\0&1&1\\1&1&0\end{bmatrix} = \begin{bmatrix}3&2&1\\1&2&1\end{bmatrix}$$

M $\quad \mathbf{n}_1\ \mathbf{n}_2\ \mathbf{n}_3 \quad M\mathbf{n}_1\ M\mathbf{n}_2\ M\mathbf{n}_3$

by rows: N transforms each row of M

$$\begin{matrix}\mathbf{r}_1\\ \mathbf{r}_2\end{matrix}\begin{bmatrix}1&0&2\\0&1&1\end{bmatrix} \times \begin{bmatrix}1&0&1\\0&1&1\\1&1&0\end{bmatrix} = \begin{matrix}\mathbf{r}_1N\\ \mathbf{r}_2N\end{matrix}\begin{bmatrix}3&2&1\\1&2&1\end{bmatrix}$$

N



$(MN)_{:,j} = M\mathbf{n}_j$

M transforms each column $\mathbf{n}_j$ of N.

$(MN)_{i,:} = \mathbf{r}_i N$

N transforms each row $\mathbf{r}_i$ of M.

Composition of Linear Transformations

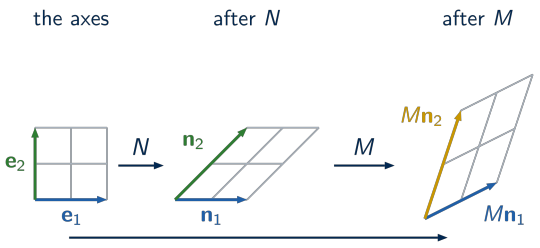
Multiplying matrices composes their transformations: the product MN applies N first and then M.

$(MN)\mathbf{x} = M(N\mathbf{x})$

Transforming by N and then by M is a single transformation, and its matrix is the product. Products are associative, $(MN)K = M(NK)$, so the parentheses can go anywhere.

$(MN)\mathbf{e}_j = M\mathbf{n}_j$

N sends the axis $\mathbf{e}_j$ to its column $\mathbf{n}_j$, and M sends that to $M\mathbf{n}_j$, column j of MN. The figure follows $\mathbf{e}_2$ to the gold $M\mathbf{n}_2$.

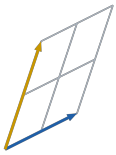


Rules for Matrix Operations

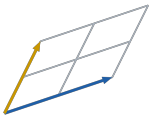
Addition and the Hadamard product commute, but the matrix product does not.

addition commutes	$M + N = N + M$
Hadamard product commutes	$M \odot N = N \odot M$
matrix product is associative	$(MN)K = M(NK)$
it distributes over addition	$M(N + K) = MN + MK$
	$(M + N)K = MK + NK$
it does not commute	$MN \neq NM$

MN: first N, then M



NM: first M, then N



1.0	0.5
0.5	1.5

$\neq$

1.5	0.5
0.5	1.0

MN applies N first and NM applies M first, so they are different transformations. When the shapes differ, one of the two products may not exist.

The Identity Matrix and Matrix Units

The identity matrix changes nothing, and a matrix unit picks out one entry.

$$MI = M, \qquad I\mathbf{x} = \mathbf{x}$$

The columns of I are the standard basis vectors, so column j of MI is $M\mathbf{e}_j$, column j of M.

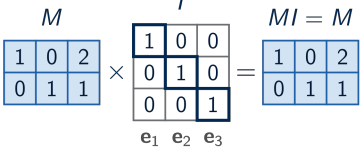
$$E_{ij} = \mathbf{e}_i\mathbf{e}_j^{\mathsf{T}}, \qquad \mathbf{e}_i^{\mathsf{T}} M \mathbf{e}_j = M_{ij}$$

The matrix unit E_{ij} has a single 1, at row i and column j.

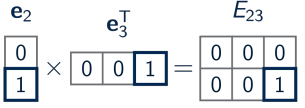
$$M = \sum_{i,j} M_{ij} E_{ij}, \qquad I = \sum_i E_{ii}$$

The matrix units are the standard basis of the $m \times n$ matrices.

the identity matrix



a matrix unit



$\mathbf{e}_2^{\mathsf{T}} M \mathbf{e}_3 = M_{23} = 1$
picks out entry $(2, 3)$

$$M = E_{11} + 2E_{13} + E_{22} + E_{23}$$

Moving a Matrix Across an Inner Product

A matrix can move from one argument of an inner product to the other by being transposed.

$$(MN)^{\mathsf{T}} = N^{\mathsf{T}} M^{\mathsf{T}}$$

Transposing a product transposes each factor and reverses their order, so the shapes still fit.

$$\langle \mathbf{u}, M\mathbf{x} \rangle = \langle M^{\mathsf{T}}\mathbf{u}, \mathbf{x} \rangle$$

Both sides add the same products $u_i M_{ij} x_j$. So M can act on $\mathbf{x}$, or it can act on $\mathbf{u}$ after being transposed, and the inner product stays the same.

each factor transposes, and the order reverses

$$(MN)^{\mathsf{T}} = N^{\mathsf{T}} M^{\mathsf{T}}$$

M moves across, transposed

$$\langle \mathbf{u},\ M\mathbf{x} \rangle = \langle M^{\mathsf{T}}\mathbf{u},\ \mathbf{x} \rangle$$

$\langle [1,-1],\ [5,2] \rangle = 5 - 2 = 3$

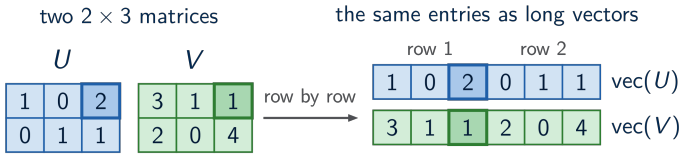
$\langle [1,-1,1],\ [1,0,2] \rangle = 1 + 0 + 2 = 3$

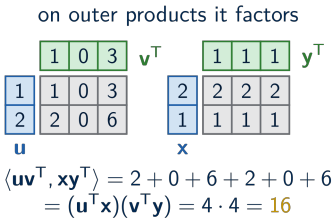
M is 2×3, $\mathbf{x} \in \mathbb{R}^3$, $\mathbf{u} = [1,-1]^{\mathsf{T}} \in \mathbb{R}^2$

The Frobenius Inner Product

The Frobenius Inner Product

The Frobenius inner product multiplies matching entries of two arrays of the same shape and adds them all.





$$\langle \mathcal{A}, \mathcal{B} \rangle = \sum_{i_1,\ldots,i_k} \mathcal{A}_{i_1\ldots i_k} \mathcal{B}_{i_1\ldots i_k}$$

Every index is summed; for matrices, that adds the inner products of matching rows.

$$\langle A,\ \mathbf{u}\mathbf{v}^\mathsf{T} \rangle = \mathbf{u}^\mathsf{T} A\, \mathbf{v}$$

When $A = \mathbf{x}\mathbf{y}^\mathsf{T}$ too, it is $(\mathbf{u}^\mathsf{T}\mathbf{x})(\mathbf{v}^\mathsf{T}\mathbf{y})$, a product of the vectors' inner products.

Lengths and Angles Between Matrices

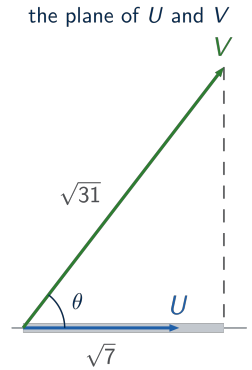
The Frobenius inner product gives matrices lengths and angles, so two matrices are two arrows in the plane they span.

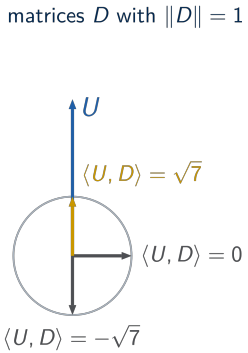
$\|U\|^2 = \langle U, U \rangle$

The length of a matrix is its Frobenius norm, the square root of the sum of its squared entries.

$\langle U, V \rangle = \|U\| \, \|V\| \cos\theta$

As with vectors, for V of a fixed length it is largest when V points along U; it is zero when they are perpendicular and negative when they point apart.





Linearity of the Frobenius Inner Product

The Frobenius inner product is linear in each argument, and pairing with a matrix unit picks out one entry.

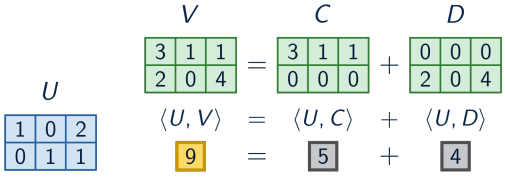
$$\langle U,\ C + D\rangle = \langle U, C\rangle + \langle U, D\rangle$$

A sum in either argument splits into separate inner products, a scalar comes out, and the arguments can trade places, $\langle U, V\rangle = \langle V, U\rangle$, or both be transposed, $\langle U^{\mathsf{T}}, V^{\mathsf{T}}\rangle = \langle U, V\rangle$, since that pairs the same entries.

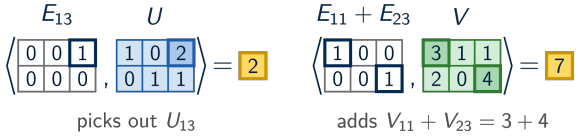
$$\langle E_{ij},\ U\rangle = U_{ij}$$

Pairing with the matrix unit E_{ij} keeps only entry (i, j), and a sum of matrix units adds up the entries they mark.

a sum splits



a matrix unit picks out an entry



Linear Functions of Matrices

Every linear function of a matrix is a Frobenius inner product with one constant matrix.

$$L(D) = \langle G, D \rangle, \qquad G_{ij} = L(E_{ij})$$

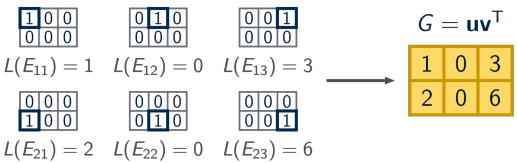
Applying L to each matrix unit fills in G. Linearity does the rest, since every matrix is a sum of its entries times matrix units.

$$\mathbf{u}^\mathsf{T} D\, \mathbf{v} = \langle \mathbf{u}\mathbf{v}^\mathsf{T}, D \rangle$$

Putting D between two vectors is such a function, and its G is their outer product.

apply $L(D) = \mathbf{u}^\mathsf{T} D\, \mathbf{v}$ to each matrix unit

$\mathbf{u} = (1, 2)$, $\mathbf{v} = (1, 0, 3)$, so $L(E_{ij}) = u_i v_j$



check on one matrix U

U

1	0	2
0	1	1

$$U\mathbf{v} = (7, 3)$$
$$\mathbf{u}^\mathsf{T} U \mathbf{v} = 1 \cdot 7 + 2 \cdot 3 = 13$$
$$\langle G, U \rangle = 1 + 0 + 6 + 0 + 0 + 6 = 13$$

The Trace and the Frobenius Inner Product

The Frobenius inner product of two matrices is a trace.

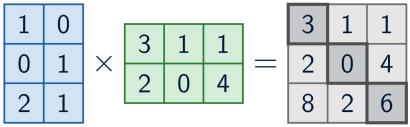
$$\operatorname{tr}(M) = \sum_i M_{ii}$$

The trace of a square matrix adds the entries on its diagonal.

$$\langle U, V \rangle = \operatorname{tr}(U^\mathsf{T} V)$$

Entry (j, j) of $U^\mathsf{T} V$ is the inner product of column j of U with column j of V, so the trace adds every product of matching entries once.

the diagonal of $U^\mathsf{T} V$



U^T V $U^\mathsf{T} V$

$$(U^\mathsf{T} V)_{11} = \langle (1,0), (3,2) \rangle = 3$$
$$(U^\mathsf{T} V)_{22} = \langle (0,1), (1,0) \rangle = 0$$
$$(U^\mathsf{T} V)_{33} = \langle (2,1), (1,4) \rangle = 6$$

$$\operatorname{tr}(U^\mathsf{T} V) = 3 + 0 + 6 = 9 = \langle U, V \rangle$$

Cyclic Permutations of the Trace

Rotating the factors of a product does not change its trace.

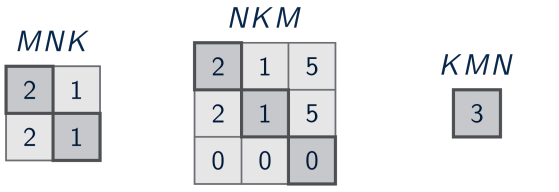
$$\mathrm{tr}(MNK) = \mathrm{tr}(NKM) = \mathrm{tr}(KMN)$$

The three products can have different shapes, as long as each one is square.

$$\mathrm{tr}(MNK) = \sum_{i,j,k} M_{ij} N_{jk} K_{ki}$$

The indices link M to N to K and back to M in a closed loop, so a rotation only starts reading the loop at a different factor.

three rotations, three shapes, one trace



$2\times3 \cdot 3\times1 \cdot 1\times2$ $\quad$ $3\times1 \cdot 1\times2 \cdot 2\times3$ $\quad$ $1\times2 \cdot 2\times3 \cdot 3\times1$

$\mathrm{tr} = 2 + 1 = 3$ $\quad$ $\mathrm{tr} = 2 + 1 + 0 = 3$ $\quad$ $\mathrm{tr} = 3$

Moving Factors Across the Frobenius Inner Product

Inside a Frobenius inner product, a factor can move from one argument to the other: a matrix factor is transposed, and a Hadamard factor is not.

$\langle G, AW\rangle = \langle A^{\mathsf{T}}G,\ W\rangle$

Both sides equal $\mathrm{tr}(G^{\mathsf{T}}AW)$, so the left factor A moves next to G, transposed.

$\langle G, AW\rangle = \langle GW^{\mathsf{T}},\ A\rangle$

Rotating the trace to $\mathrm{tr}(WG^{\mathsf{T}}A)$ moves the right factor W next to G, transposed.

$\langle G, U \odot V\rangle = \langle U \odot G,\ V\rangle$

There is an exception for the Hadamard product: U moves without a transpose.

the shaded factor moves to the other argument — shapes

$\langle G,\ AW\rangle = \langle A^{\mathsf{T}}G,\ W\rangle$ — a matrix on the left, $B\times k \to m\times k$

$\langle G,\ AW\rangle = \langle GW^{\mathsf{T}},\ A\rangle$ — a matrix on the right, $B\times k \to B\times m$

$\langle G,\ \mathbf{a}\mathbf{b}^{\mathsf{T}}\rangle = \langle \mathbf{a}^{\mathsf{T}}G,\ \mathbf{b}^{\mathsf{T}}\rangle$ — a vector, $B\times k \to 1\times k$

$\langle G,\ U\odot V\rangle = \langle U\odot G,\ V\rangle$

an exception for the Hadamard product: no transpose

the same rules hold when G is a vector or a tensor

Tensor Products

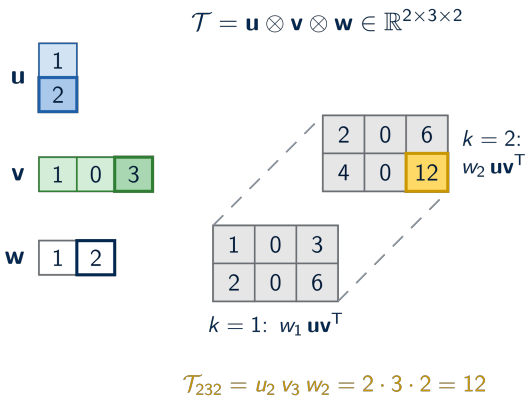
The tensor product multiplies one entry from each vector, so three vectors fill an array with three axes.

$$(\mathbf{u} \otimes \mathbf{v} \otimes \mathbf{w})_{ijk} = u_i v_j w_k$$

Every combination of entries gets its own position, and nothing is added up, so all three indices stay free. The gold entry is $2 \cdot 3 \cdot 2 = 12$.

$$\mathbf{u} \otimes \mathbf{v} = \mathbf{u}\mathbf{v}^{\mathsf{T}}$$

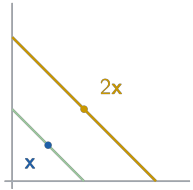
With two vectors this is the outer product. Each slice of the three-axis tensor is that outer product, scaled by one entry of $\mathbf{w}$.



Polynomial Forms

An order-k tensor with the same vector in every index is a polynomial of degree k.

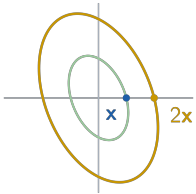
linear, degree 1



$f(\mathbf{x}) = \mathbf{a}^{\mathsf{T}}\mathbf{x} = \langle \mathbf{a}, \mathbf{x} \rangle$

order 1: $f(2\mathbf{x}) = 2\, f(\mathbf{x})$

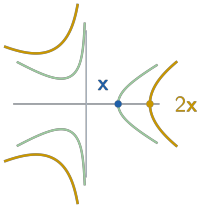
quadratic, degree 2



$q(\mathbf{x}) = \mathbf{x}^{\mathsf{T}} A\, \mathbf{x} = \langle A, \mathbf{x} \otimes \mathbf{x} \rangle$

order 2: $q(2\mathbf{x}) = 4\, q(\mathbf{x})$

cubic, degree 3



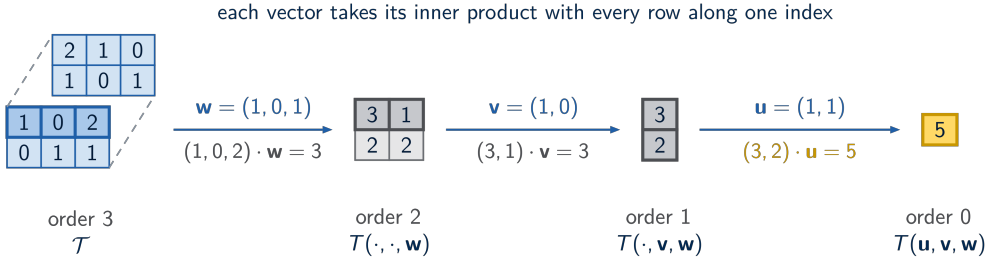
$T(\mathbf{x}) = \langle \mathcal{T}, \mathbf{x} \otimes \mathbf{x} \otimes \mathbf{x} \rangle$

order 3: $T(2\mathbf{x}) = 8\, T(\mathbf{x})$

Each term of an order-k form multiplies k entries of $\mathbf{x}$, so $T(2\mathbf{x}) = 2^k\, T(\mathbf{x})$, and every level set is a scaled copy of one shape.

Multilinear Forms

Giving each index its own vector turns a polynomial form into a multilinear form.



$\langle A, \mathbf{x} \otimes \mathbf{x} \rangle \;\rightarrow\; \langle A, \mathbf{u} \otimes \mathbf{v} \rangle$

The quadratic form becomes a bilinear form, linear in $\mathbf{u}$ and in $\mathbf{v}$ separately.

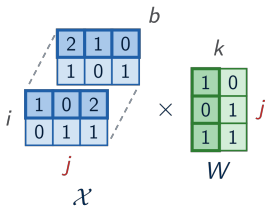
$T(\mathbf{u}, \mathbf{v}, \mathbf{w}) = \langle \mathcal{T}, \mathbf{u} \otimes \mathbf{v} \otimes \mathbf{w} \rangle$

Each vector takes its inner product with every row along its index.

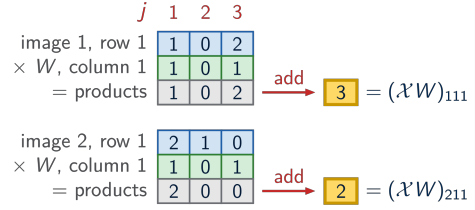
Tensor Contraction

A contraction multiplies two arrays along a shared axis, then adds across it.

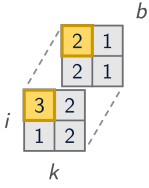
$\mathcal{X}$ and W share the axis j



for each image b: multiply along j, then add



$\mathcal{X}W$: the axes b, i, k remain



$$(\mathcal{X}W)_{bik} = \sum_j \mathcal{X}_{bij} W_{jk}$$

Row i of image b and column k of W are multiplied along j; adding removes the j axis.

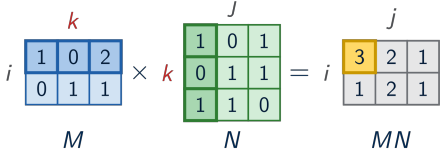
$$(MN)_{ij} = \sum_k M_{ik} N_{kj}$$

A matrix product contracts the one axis two matrices share.

Summation Notation

A shared index is summed, and the indices left over are the axes of the result.

one entry of the matrix product



$$(MN)_{11} = M_{11}N_{11} + M_{12}N_{21} + M_{13}N_{31}$$
$$= 1 \cdot 1 + 0 \cdot 0 + 2 \cdot 1 = 3$$

reading the formula

$$(MN)_{ij} = \sum_k M_{ik} N_{kj}$$

$(MN)_{ij}$ one entry of the result: the free indices i and j become the axes of MN

$\sum_k$ runs the summed index k through all its values and adds the products

$M_{ik}N_{kj}$ k appears in both factors, so row i of M is paired with column j of N

Fix the free indices to pick one entry, then run each summed index through all its values and add the products. Each term is a product of scalars, so the indices, not the order of the factors, say which entries meet.

Operations in Summation Notation

Written with indices, every operation shows which indices it sums and which it keeps.

operation	written with indices	summed	left over	
transpose	$(M^{\mathsf{T}})_{ij} = M_{ji}$	none	i, j	the indices swap
Hadamard product	$(U \odot V)_{ij} = U_{ij} V_{ij}$	none	i, j	shared, not summed
matrix-vector product	$(M\mathbf{x})_i = \sum_k M_{ik} x_k$	k	i	a vector
bilinear form	$\langle \mathbf{u}, M\mathbf{x} \rangle = \sum_{i,j} u_i M_{ij} x_j$	i, j	none	sum i first: $\langle M^{\mathsf{T}}\mathbf{u}, \mathbf{x} \rangle$
trace	$\mathrm{tr}(M) = \sum_i M_{ii}$	i	none	a scalar
composition	$(MNK)_{il} = \sum_{j,k} M_{ij} N_{jk} K_{kl}$	j, k	i, l	either sum first: $(MN)K = M(NK)$
tensor product	$(\mathbf{u} \otimes \mathbf{v} \otimes \mathbf{w})_{ijk} = u_i v_j w_k$	none	i, j, k	an order-3 tensor
multilinear form	$T(\mathbf{u}, \mathbf{v}, \mathbf{w}) = \sum_{i,j,k} \mathcal{T}_{ijk} u_i v_j w_k$	i, j, k	none	a scalar

Differentials

Partial Derivatives

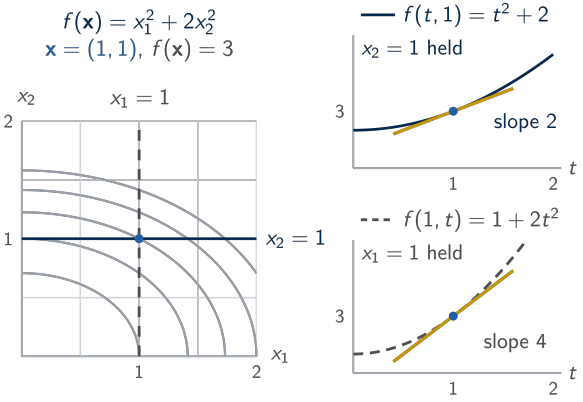
Holding all but one variable fixed turns f into a function of one variable, and its slope is a partial derivative.

$$\frac{\partial f}{\partial x_1} = \frac{d}{dt} f(t,\, x_2)\Big|_{t=x_1}$$

Move through $\mathbf{x}$ along the x_1 direction with x_2 held fixed, and take the ordinary derivative of this slice.

$$\frac{\partial f}{\partial x_1} = 2x_1, \qquad \frac{\partial f}{\partial x_2} = 4x_2$$

Differentiate as usual, treating the other variable as a constant. At $\mathbf{x}$, f rises twice as fast along x_2 as along x_1.



The Differential Is a Linear Functional

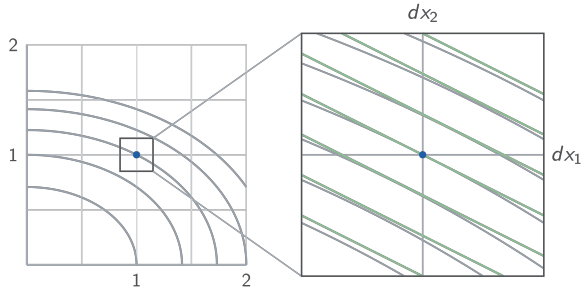
Near a point, a function changes by its differential df, a linear functional of $d\mathbf{x}$.

$$d\mathbf{x} = \begin{bmatrix} dx_1 \\ dx_2 \end{bmatrix}$$

The differential of $\mathbf{x}$ is a small displacement: the new point minus $\mathbf{x}$, a vector of the same shape. In the zoom, its entries are the axes.

$$df = \frac{\partial f}{\partial x_1}\, dx_1 + \frac{\partial f}{\partial x_2}\, dx_2$$
$$= \begin{bmatrix} \frac{\partial f}{\partial x_1} & \frac{\partial f}{\partial x_2} \end{bmatrix} d\mathbf{x}$$

Each partial derivative scales one entry of $d\mathbf{x}$, and the results add.



$f = x_1^2 + 2x_2^2$
$\mathbf{x} = (1, 1)$

$df = 2\, dx_1 + 4\, dx_2$
$= \begin{bmatrix} 2 & 4 \end{bmatrix} d\mathbf{x}$

The Gradient Vector

Transposing the row of partial derivatives gives the gradient, a vector that points uphill across the level sets.

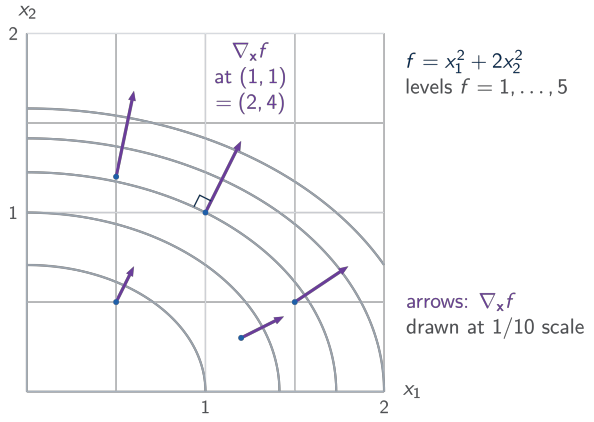
$$\nabla_{\mathbf{x}} f = \begin{bmatrix} \partial f/\partial x_1 \\ \partial f/\partial x_2 \end{bmatrix}$$

$$df = \langle \nabla_{\mathbf{x}} f,\, d\mathbf{x} \rangle$$

The subscript names the variable; the other variables are held fixed.

$$\nabla_{\mathbf{x}} f(\mathbf{x}) = \begin{bmatrix} 2x_1 \\ 4x_2 \end{bmatrix}$$

A level set is a curve where f is constant. The gradient crosses each at a right angle and is longer where they lie closer.



How a Gradient Acts on a Differential

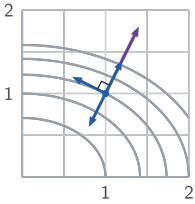
The gradient is paired with a differential $d\mathbf{x}$, not with the point $\mathbf{x}$ itself: their inner product is the differential df.

$$df = \langle \nabla_{\mathbf{x}} f,\, d\mathbf{x} \rangle = \|\nabla_{\mathbf{x}} f\|\, \|d\mathbf{x}\| \cos\theta$$

Among differentials $d\mathbf{x}$ of one length, f rises fastest along the gradient, not at all along a level set, and falls fastest against it.

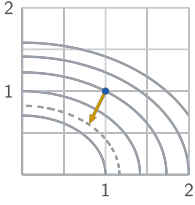
$$\mathbf{x} \leftarrow \mathbf{x} - \eta\, \nabla_{\mathbf{x}} f$$

Moving against the gradient lowers f fastest, and the learning rate η sets the step size. Repeating this update is gradient descent.



$f = x_1^2 + 2x_2^2$ at $\mathbf{x} = (1, 1)$
$\nabla_{\mathbf{x}} f = (2, 4)$, drawn at $\frac{1}{5}$ length

$d\mathbf{x}$	$df/\|d\mathbf{x}\|$
along $\nabla_{\mathbf{x}} f$	$+4.47$
along the level set	0
against $\nabla_{\mathbf{x}} f$	-4.47



one gradient descent update, $\eta = 0.1$

$$\mathbf{x} - \eta\, \nabla_{\mathbf{x}} f = (1, 1) - 0.1\,(2, 4) = (0.8,\, 0.6)$$

f falls from 3 to 1.36
dashed: the level $f = 1.36$

Gradients of Linear and Quadratic Functions

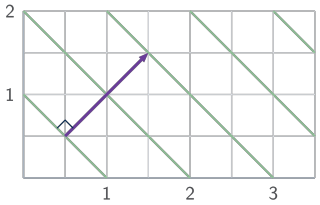
With respect to $\mathbf{x}$, the gradient of $\mathbf{w}^T\mathbf{x}$ is $\mathbf{w}$ at every point, and the gradient of $\mathbf{x}^T\mathbf{x}$ is $2\mathbf{x}$.

$$d(\mathbf{w}^T\mathbf{x}) = \mathbf{w}^T d\mathbf{x} \;\Rightarrow\; \nabla_{\mathbf{x}}(\mathbf{w}^T\mathbf{x}) = \mathbf{w}$$

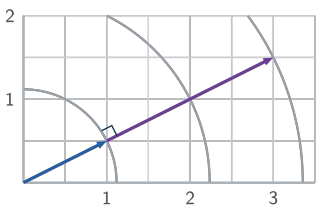
The levels are parallel lines, and the gradient is the same arrow everywhere.

$$d(\mathbf{x}^T\mathbf{x}) = 2\,\mathbf{x}^T d\mathbf{x} \;\Rightarrow\; \nabla_{\mathbf{x}}(\mathbf{x}^T\mathbf{x}) = 2\mathbf{x}$$

The levels are circles, and the gradient points straight out from the origin, longer farther out.



$f = \mathbf{w}^T\mathbf{x}$
$\nabla_{\mathbf{x}} f = \mathbf{w} = (1, 1)$
levels of f: parallel lines



$f = \mathbf{x}^T\mathbf{x}$
$\mathbf{x} = (1, 0.5)$
$\nabla_{\mathbf{x}} f = 2\mathbf{x} = (2, 1)$
levels of f: circles

Matrix Gradients

For a function of a matrix, the gradient is the matrix of partial derivatives, and the differential is its Frobenius inner product with dN.

$$(\nabla_N f)_{ij} = \frac{\partial f}{\partial N_{ij}}$$

One partial derivative for each entry, so the gradient has the shape of N.

$$df = \sum_{i,j} \frac{\partial f}{\partial N_{ij}}\, dN_{ij} = \langle \nabla_N f,\, dN \rangle$$

The differential dN is the new matrix minus N, so it has the shape of N. Adding each partial derivative times dN_{ij} gives df; for $f(N) = \langle C, N \rangle$ the gradient is C.

$f(N) = \langle C, N \rangle$ with $C = \left[\begin{smallmatrix} 1 & 2 \\ 0 & 1 \end{smallmatrix}\right]$

N

1	0
2	1

nudge entry $(1,2)$ by a small h → f changes by $2h$

why: the inner product picks out one entry of the gradient

$$df = \left\langle \begin{array}{|c|c|}\hline 1 & 2 \\ \hline 0 & 1 \\ \hline \end{array} \;,\; \begin{array}{|c|c|}\hline 0 & h \\ \hline 0 & 0 \\ \hline \end{array} \right\rangle = 2h$$

$\nabla_N f = C$ $\qquad$ dN

Rules for Differentials

A linear operation passes the differential straight through, and a product of two varying matrices gives one term per factor, each in its place.

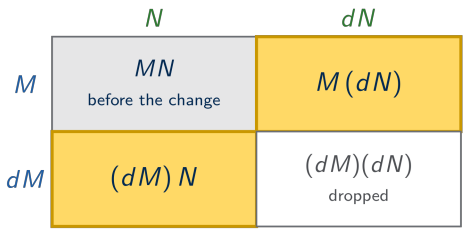
$L(U + dU) - L(U) = L(dU)$

For a linear L the change is exactly $L(dU)$, so $d(U + V) = dU + dV$, $d(U^{\mathsf{T}}) = (dU)^{\mathsf{T}}$ and $d\,\mathrm{tr}(U) = \mathrm{tr}(dU)$. A constant matrix does not change, so $dC = 0$.

$d(MN) = (dM)\,N + M\,(dN)$

Expanding $(M + dM)(N + dN)$ gives one product per pair; $(dM)(dN)$, a product of two differentials, is dropped, and $(dM)\,N$ cannot become $N\,(dM)$.

$(M + dM)(N + dN)$, one product per pair



$$d(MN) = (dM)\,N + M\,(dN)$$

the factor from M stays on the left, the one from N on the right

Differentials of Element-wise Functions

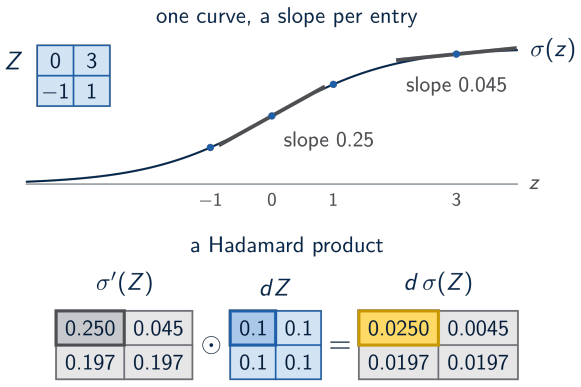
An element-wise function multiplies each entry's differential by the derivative at that entry, so its differential is a Hadamard product.

$$d\,f(Z) = f'(Z) \odot dZ$$

Each entry moves along its own copy of the curve, so the slope there scales its differential.

$$\sigma'(z) = \sigma(z)\left(1 - \sigma(z)\right)$$

For the sigmoid $\sigma(z) = 1/(1+e^{-z})$, the slope peaks at 0 and nearly vanishes for large $|z|$, so an entry there is saturated: it barely changes.



Rules of Matrix Differentials

Each rule gives the differential of one operation, and nesting them gives the differential of any expression.

linear operations

constant matrix	$dC = 0$
sum	$d(U + V) = dU + dV$
constant factor	$d(CU) = C\,dU$
	$d(UC) = (dU)\,C$
outer product with $\mathbf{1}$	$d(\mathbf{1}\mathbf{b}^\mathsf{T}) = \mathbf{1}\,d\mathbf{b}^\mathsf{T}$
transpose	$d(U^\mathsf{T}) = (dU)^\mathsf{T}$
trace	$d\,\mathrm{tr}(U) = \mathrm{tr}(dU)$

products of two varying matrices

matrix product	$d(UV) = (dU)\,V + U\,dV$
Hadamard product	$d(U \odot V) = dU \odot V + U \odot dV$
Frobenius inner product	$d\langle U, V\rangle = \langle dU, V\rangle + \langle U, dV\rangle$
squared Frobenius norm	$d\lVert U\rVert^2 = 2\langle U, dU\rangle$

functions

element-wise function	$d\sigma(U) = \sigma'(U) \odot dU$
chain rule, $f = g(S)$	$df = \langle \nabla g(S), dS\rangle$

Rules nest: in $d\sigma(MN) = \sigma'(MN) \odot d(MN)$, the product rule then expands $d(MN)$.

Reading Off a Gradient

A matrix G equals $\nabla_N f$ if and only if $df = \langle G, dN \rangle$ for every dN.

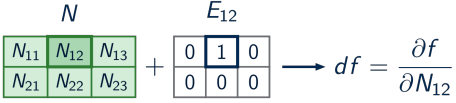
$$\begin{aligned} df &= \textstyle\sum_{i,j} \frac{\partial f}{\partial N_{ij}}\, dN_{ij} \\ &= \textstyle\sum_{i,j} (\nabla_N f)_{ij}\, dN_{ij} \\ &= \langle \nabla_N f,\, dN \rangle \end{aligned}$$

The partial derivatives are the entries of $\nabla_N f$, and adding products of matching entries is the Frobenius inner product.

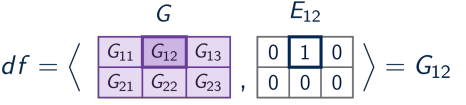
$$G_{ij} = \langle G, E_{ij} \rangle = \langle \nabla_N f, E_{ij} \rangle = \frac{\partial f}{\partial N_{ij}}$$

Both inner products are df at $dN = E_{ij}$, which picks out entry (i, j), so the gradient is read off without computing any partial derivative.

$dN = E_{12}$ moves only entry $(1, 2)$ of N



the inner product with E_{12} picks out entry $(1, 2)$ of G



$$G_{12} = \frac{\partial f}{\partial N_{12}}$$

and for every entry: $G = \nabla_N f$

Reading Off a Gradient Through a Product

Given $f = g(S)$, $S = MN$, a constant matrix M and a scalar function g, find $\nabla_N f$.

given	$f = g(S)$
differential of g at S (chain rule)	$df = \sum_{i,j} \frac{\partial g}{\partial S_{ij}}\, dS_{ij}$
$G = \nabla g(S)$ has entries $\partial g / \partial S_{ij}$	$= \langle G,\, dS \rangle$
$S = MN$	$= \langle G,\, d(MN) \rangle$
product rule	$= \langle G,\, (dM)\, N + M\, dN \rangle$
linearity in the second argument	$= \langle G,\, (dM)\, N \rangle + \langle G,\, M\, dN \rangle$
M is a constant matrix, so $dM = 0$	$= \langle G,\, 0\, N \rangle + \langle G,\, M\, dN \rangle$
$0\, N$ is the zero matrix	$= \langle G, 0 \rangle + \langle G,\, M\, dN \rangle$
$\langle G, 0 \rangle = 0$	$= \langle G,\, M\, dN \rangle$
move M across, transposed	$= \langle M^{\mathsf{T}} G,\, dN \rangle$
both are df for every dN, and $dN = E_{ij}$ picks out entry (i, j)	$\nabla_N f = \nabla_N g(S) = M^{\mathsf{T}} G$

with numbers

M^{T} G $\nabla_N f$

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 2 & 1 \end{bmatrix} \times \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 0 & 1 \\ 2 & 5 \end{bmatrix}$$

M is 2×3 and N is 3×2, so S and G are 2×2; $M^{\mathsf{T}} G$ is 3×2, the shape of N

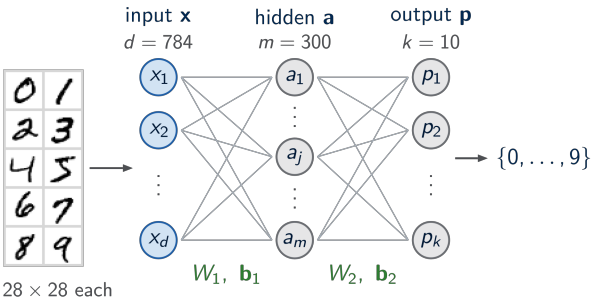
g must return a scalar: only a scalar has a gradient. For any other $S = F(N)$, write dS in terms of dN with the rules, then move factors until dN stands alone.

The Forward Pass

The Network as Neurons and as a Function

One network, written two ways: as neurons and as one function.

as neurons



as one function of one example

$$\mathbf{p} = \text{softmax}\big(W_2^{\mathsf{T}}\, \sigma(W_1^{\mathsf{T}}\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2\big)$$

$$\ell = -\ln p_y$$

The neurons show the network's shape, and the function shows what it computes.

The Network for a Batch

For a batch of B examples, the network is one function of the data, computed by five assignments in order.

as assignments, one step each

first layer	$XW_1 + \mathbf{1}\mathbf{b}_1^{\mathsf{T}} \to Z_1$	$B \times m$
hidden layer	$\sigma(Z_1) \to A$	$B \times m$
logits	$AW_2 + \mathbf{1}\mathbf{b}_2^{\mathsf{T}} \to T$	$B \times k$
probabilities	$\mathrm{softmax}(T) \to P$	$B \times k$
loss	$-\frac{1}{B}\langle Y, \ln P \rangle \to J$	scalar

as one function of a batch of B examples

$$P = \mathrm{softmax}\big(\sigma(XW_1 + \mathbf{1}\mathbf{b}_1^{\mathsf{T}})\, W_2 + \mathbf{1}\mathbf{b}_2^{\mathsf{T}}\big)$$

$$J = -\tfrac{1}{B}\langle Y, \ln P \rangle$$

$X \in \mathbb{R}^{B \times d}$: one image in each row

$Y \in \{0,1\}^{B \times k}$: its one-hot label in the same row

Each assignment computes one array from the arrays before it, and its arrow points to the array it makes.

Vectorization

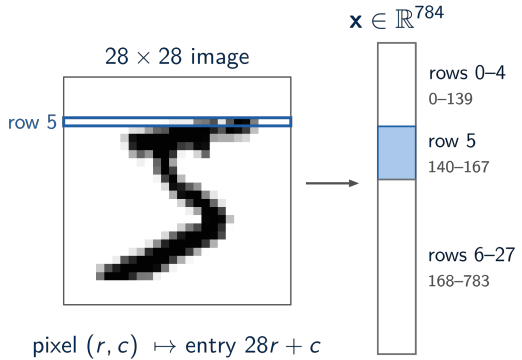
To give an image to a network, we read its pixels row by row into one long vector.

$(r, c) \;\mapsto\; 28r + c$

Counting from zero, the pixel in row r and column c becomes entry $28r + c$. Row 0 fills entries 0 to 27, row 1 fills entries 28 to 55, and so on.

$\mathbf{x} \in \mathbb{R}^{784}$

The 784 values do not change, only their arrangement. This is the same row-by-row order as vec.



Data Matrices

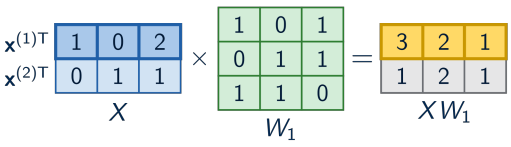
A data matrix stores one example in each row, so it is a point cloud that one matrix product moves at once.

$X \in \mathbb{R}^{B \times d}$

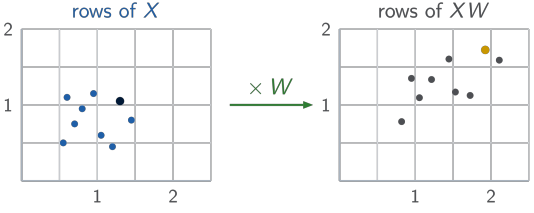
Row b of X is example b, a point in $\mathbb{R}^d$, so the B rows of X form a point cloud.

row b of $XW_1 = \left(W_1^\top \mathbf{x}^{(b)}\right)^\top$

Row b of XW_1 is example b after the linear transformation, so the whole point cloud moves through the same map at once.



each row is a point, and one product moves them all



Activation Functions

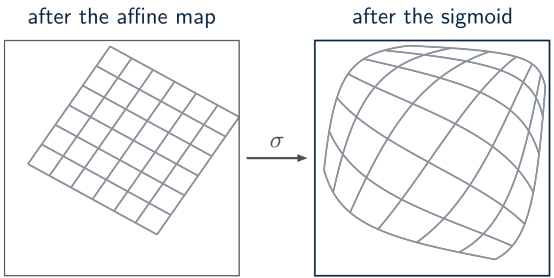
Without an activation function between them, two layers can do no more than one layer.

$$W_2^{\mathsf{T}}(W_1^{\mathsf{T}}\mathbf{x}+\mathbf{b}_1)+\mathbf{b}_2 = (W_1 W_2)^{\mathsf{T}}\mathbf{x}+\mathbf{c}$$

Each layer is a linear map plus a bias, which is called an affine map. One affine map followed by another is again an affine map, so a second layer would add nothing new.

$$\sigma : \mathbb{R}^m \to (0,1)^m$$

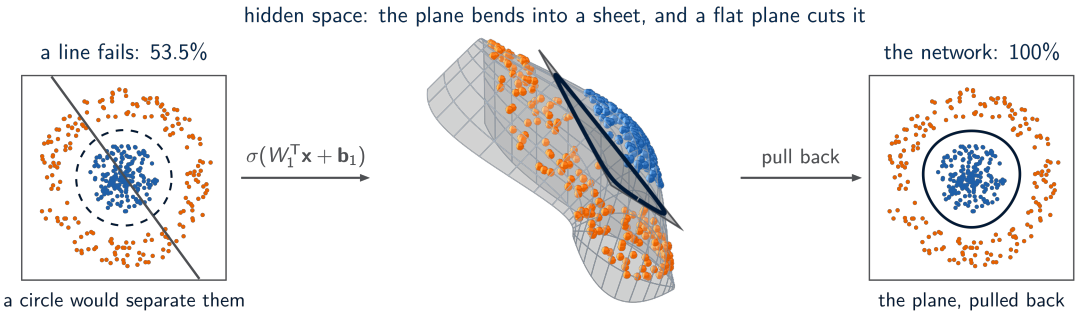
The sigmoid bends the straight grid lines of an affine map into curves.



a square grid of inputs through $W^{\mathsf{T}}\mathbf{x}+\mathbf{b}$, then σ

Linear Separability

A hidden layer bends the plane so that a flat plane separates the classes.



No line separates the disc from the ring, so a linear classifier reaches only 53.5 %. The hidden layer $\mathbf{h} = \sigma(W_1^{\mathsf{T}}\mathbf{x} + \mathbf{b}_1)$ bends the plane into a sheet in $\mathbb{R}^3$, where the flat plane $h_1 + h_2 + h_3 = \tau$ separates the classes.

Logits

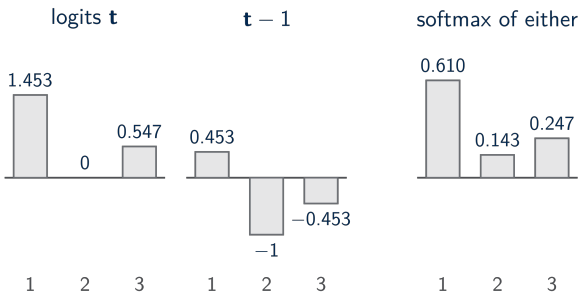
The logits are the output layer's raw scores, one per class.

$$T = AW_2 + \mathbf{1}\mathbf{b}_2^{\mathsf{T}} \in \mathbb{R}^{B \times k}$$

Row b holds example b's k logits, one per class. They can be any size or sign, and they do not add up to 1.

$$P_{bc} \propto \exp(T_{bc})$$

Each probability grows with its logit, so a larger logit means a more likely class. Adding the same number to every logit in a row multiplies each $\exp(T_{bc})$ by the same factor, so the probabilities do not change.



The Softmax Function

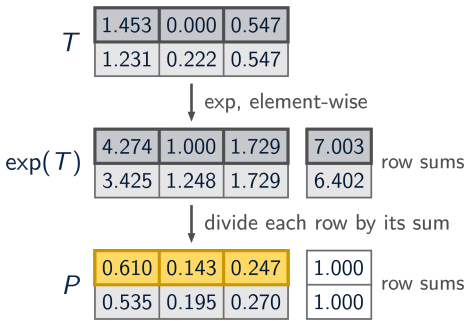
The output layer gives each example one row of logits, and the softmax turns each row into predicted probabilities that add up to 1.

$$T = AW_2 + \mathbf{1}\mathbf{b}_2^{\mathsf{T}}$$

The output layer is an affine map of the hidden layer A, and its outputs are the logits, one row per example and one column per class.

$$P_{bc} = \frac{\exp(T_{bc})}{\sum_{c'} \exp(T_{bc'})}$$

The exponential makes every entry positive, and dividing each row by its own sum makes that row add up to 1.



row 1: $4.274 + 1.000 + 1.729 = 7.003$

$(4.274,\ 1.000,\ 1.729)/7.003 = (0.610,\ 0.143,\ 0.247)$

Vectorizing the Softmax

Each step of the softmax is one matrix operation on the whole batch.

operation	one entry at a time	the whole batch at once	result
exponential, element-wise	$E_{bc} = \exp(T_{bc})$	$E = \exp(T)$	4.274 1.000 1.729 3.425 1.248 1.729
add up each row	$u_b = \sum_c E_{bc}$	$\mathbf{u} = E\,\mathbf{1}$	7.003 6.402
broadcast each row's sum	u_b for every c	$\mathbf{u}\,\mathbf{1}^\mathsf{T}$	7.003 7.003 7.003 6.402 6.402 6.402
Hadamard division	$P_{bc} = E_{bc}/u_b$	$P = E \oslash (\mathbf{u}\,\mathbf{1}^\mathsf{T})$	0.610 0.143 0.247 0.535 0.195 0.270

Multiplying by the vector of ones adds up each row, and $\mathbf{u}\mathbf{1}^\mathsf{T}$ broadcasts each row's sum across that row, so the Hadamard division $\oslash$ divides every row by its own sum.

Cross-Entropy Loss

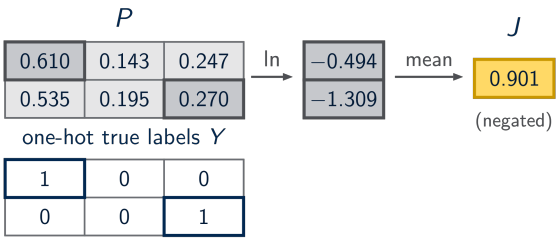
The cross-entropy loss averages the negative log-probability of the true class over the examples.

$$J = -\frac{1}{B}\sum_{b} \ln P_{b,\,y_b}$$

Here y_b is the true class of example b. The loss is near 0 when that class gets probability near 1.

$$J = -\frac{1}{B}\,\langle Y, \ln P\rangle$$

$Y \in \{0,1\}^{B\times k}$ holds the true labels, one-hot: row b has a 1 in column y_b and 0 elsewhere, so the inner product picks out the true classes.



each row's 1 marks the true class, shaded in P

$$J = -\tfrac{1}{2}\big(\ln 0.610 + \ln 0.270\big)$$
$$= -\tfrac{1}{2}(-0.494 - 1.309) = 0.901$$

Vectorizing the Cross-Entropy Loss

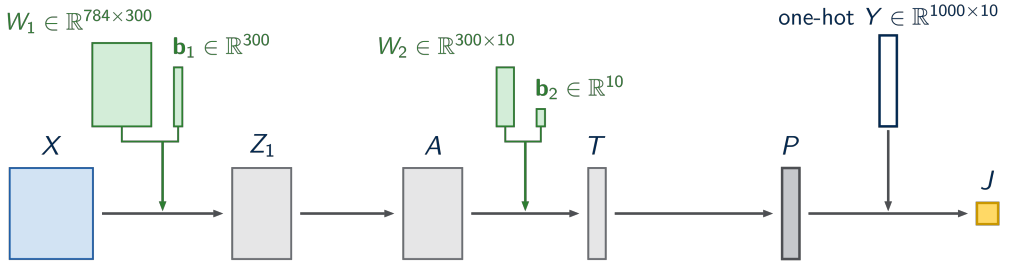
The true labels Y pick out each true class, so one inner product replaces the sum.

operation	one entry at a time	the whole batch at once	result
logarithm, element-wise	$\ln P_{bc}$	$\ln P$	$\begin{bmatrix} -0.494 & -1.946 & -1.399 \\ -0.625 & -1.635 & -1.309 \end{bmatrix}$
one-hot true labels	y_b, the true class	Y	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$
keep each true class	$\ln P_{b,\,y_b}$	$Y \odot \ln P$	$\begin{bmatrix} -0.494 & 0 & 0 \\ 0 & 0 & -1.309 \end{bmatrix}$
add up and average	$J = -\frac{1}{B}\sum_b \ln P_{b,\,y_b}$	$J = -\frac{1}{B}\langle Y, \ln P\rangle$	0.901

The Hadamard product with Y keeps $\ln P_{b,y_b}$ in row b and sets every other entry to 0, and the Frobenius inner product $\langle Y, \ln P\rangle$ adds up every entry of $Y \odot \ln P$.

Forward Propagation

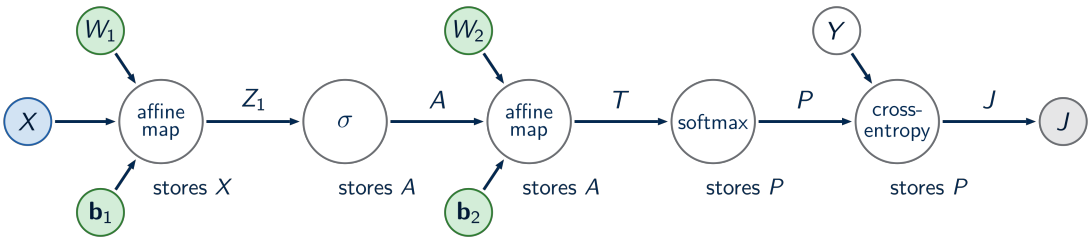
The forward pass computes the arrays in order from the data X to the loss J. The loss is a scalar that depends on the parameters W_1, $\mathbf{b}_1$, W_2 and $\mathbf{b}_2$.



$$XW_1 + \mathbf{1}\mathbf{b}_1^\mathsf{T} \to Z_1 \qquad \sigma(Z_1) \to A \qquad AW_2 + \mathbf{1}\mathbf{b}_2^\mathsf{T} \to T \qquad \text{softmax}(T) \to P \qquad -\tfrac{1}{B}\langle Y, \ln P\rangle \to J$$

From Forward Propagation to a Graph

Each assignment becomes a node, and each array becomes an edge.

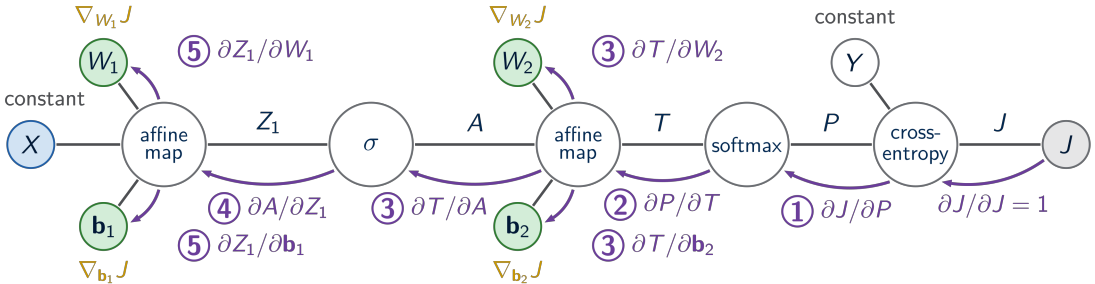


Going forward runs the nodes from left to right, so every input is ready when its node runs. Each node stores the arrays its local rule will need: X for the first affine map, A for the sigmoid and the second affine map, and P for the softmax and the loss.

The Backward Pass

Going Backward

The backward pass starts at the loss and works back to the parameters, computing gradients as it goes.



The numbers give the order of the steps. At each step, a node multiplies the gradient it receives by its local derivative and passes the result back, so a parameter's gradient is the product of the local derivatives along its path from J.

The Chain Rule Along Each Path

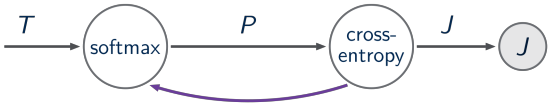
A parameter's gradient is the product of the partial derivatives on its path from J.

	step	①	②	③	④	⑤
$\frac{\partial J}{\partial W_2}$	$=$	$\frac{\partial J}{\partial P}$	$\frac{\partial P}{\partial T}$	$\frac{\partial T}{\partial W_2}$		
$\frac{\partial J}{\partial \mathbf{b}_2}$	$=$	$\frac{\partial J}{\partial P}$	$\frac{\partial P}{\partial T}$	$\frac{\partial T}{\partial \mathbf{b}_2}$		
$\frac{\partial J}{\partial W_1}$	$=$	$\frac{\partial J}{\partial P}$	$\frac{\partial P}{\partial T}$	$\frac{\partial T}{\partial A}$	$\frac{\partial A}{\partial Z_1}$	$\frac{\partial Z_1}{\partial W_1}$
$\frac{\partial J}{\partial \mathbf{b}_1}$	$=$	$\frac{\partial J}{\partial P}$	$\frac{\partial P}{\partial T}$	$\frac{\partial T}{\partial A}$	$\frac{\partial A}{\partial Z_1}$	$\frac{\partial Z_1}{\partial \mathbf{b}_1}$

The columns follow the steps of the backward pass. Every path starts with $\partial J/\partial P$ and $\partial P/\partial T$, shaded because they are computed together, and the paths to W_1 and $\mathbf{b}_1$ share their first four factors, so the backward pass computes each shared product once.

Splitting the Graph at P

Splitting at P is correct in exact arithmetic but not numerically stable.



$$\nabla_P J = -\tfrac{1}{B}\, Y \oslash P$$

$$dJ = -\tfrac{1}{B}\langle Y,\, d\ln P\rangle = -\tfrac{1}{B}\langle Y,\, dP \oslash P\rangle$$

the derivative of $\ln p$ is $1/p$: the loss divides by P

true class $y = 1$, logits $\mathbf{t} = (0,\, 800,\, 0)$

$\mathbf{p} = (0,\, 1,\, 0)$

e^{-800} is too small for floating point and rounds to 0

$\nabla_{\mathbf{p}}\ell = -\mathbf{y} \oslash \mathbf{p} = (-\infty,\, 0,\, \mathrm{NaN})$

$1/0$ and $0/0$

$\nabla_{\mathbf{t}}\ell = (\mathrm{NaN},\, \mathrm{NaN},\, \mathrm{NaN})$

the softmax's rule multiplies by $\mathbf{p}$: $0 \cdot \infty$ is not a number

Its local gradient divides by P. The softmax's rule would multiply by P and cancel the division, but in floating point P can be exactly 0, and the gradient becomes NaN.

The Differential of the Log-Probabilities

Given $P = \text{softmax}(T)$ row by row, find $d \ln P$ in terms of dT.

row sums of $\exp(T)$: the softmax denominators	$\mathbf{u} = \exp(T)\,\mathbf{1}$
softmax, then the logarithm of a quotient, row by row	$\ln P = T - (\ln \mathbf{u})\,\mathbf{1}^\mathsf{T}$
sum rule; $\mathbf{1}$ is a constant matrix	$d \ln P = dT - (d \ln \mathbf{u})\,\mathbf{1}^\mathsf{T}$
differential of an element-wise function	$d \ln \mathbf{u} = \ln'(\mathbf{u}) \odot d\mathbf{u}$
constant factor rule: $\mathbf{1}$ is a constant matrix	$= \ln'(\mathbf{u}) \odot \big(d \exp(T)\big)\,\mathbf{1}$
differential of an element-wise function	$= \ln'(\mathbf{u}) \odot \big(\exp(T) \odot dT\big)\,\mathbf{1}$
entry b of $\ln'(\mathbf{u})$ is $1/u_b$, and row b of $\exp(T)$ divided by u_b is row b of P	$= (P \odot dT)\,\mathbf{1}$
substitute $d \ln \mathbf{u}$	$d \ln P = \boxed{dT - \big((P \odot dT)\,\mathbf{1}\big)\,\mathbf{1}^\mathsf{T}}$

The Long Way: The Loss's Gradient

Given the one-hot true labels Y and the loss $J = -\frac{1}{B}\langle Y, \ln P\rangle$, find $\nabla_P J$.

cross-entropy loss of the batch	$J = -\frac{1}{B}\langle Y, \ln P\rangle$
Y is a constant matrix	$dJ = -\frac{1}{B}\langle Y, d\ln P\rangle$
differential of an element-wise function: $\ln'(p) = 1/p$	$= -\frac{1}{B}\langle Y,\ dP \oslash P\rangle$
move the Hadamard division by P across, no transpose	$= \left\langle -\frac{1}{B}\ Y \oslash P,\ dP\right\rangle$
read off	$\nabla_P J = \boxed{-\frac{1}{B}\ Y \oslash P}$

Split at P, the loss sends back $-\frac{1}{B}\ Y \oslash P$: 0 wherever Y is 0, and large where the true class has a small probability.

The Long Way: The Softmax's Gradient

Given $\nabla_P J$ and $P = \text{softmax}(T)$, write dJ in terms of dT.

definition of the gradient $\nabla_P J$

$$dJ = \langle \nabla_P J,\ dP \rangle$$

differential of an element-wise function: $P = \exp(\ln P)$

$$= \langle \nabla_P J,\ P \odot d \ln P \rangle$$

the differential of the log-probabilities

$$= \left\langle \nabla_P J,\ P \odot \left(dT - ((P \odot dT)\mathbf{1})\mathbf{1}^\mathsf{T} \right) \right\rangle$$

move P across, no transpose

$$= \left\langle P \odot \nabla_P J,\ dT - ((P \odot dT)\mathbf{1})\mathbf{1}^\mathsf{T} \right\rangle$$

linearity in the second argument

$$= \boxed{\langle P \odot \nabla_P J, dT \rangle - \langle P \odot \nabla_P J,\ ((P \odot dT)\mathbf{1})\mathbf{1}^\mathsf{T} \rangle}$$

The Long Way: The Softmax's Gradient, Continued

Given $dJ = \langle P \odot \nabla_P J, dT \rangle - \langle P \odot \nabla_P J, ((P \odot dT)\mathbf{1})\mathbf{1}^\mathsf{T} \rangle$, find $\nabla_T J$.

move $\mathbf{1}^\mathsf{T}$ across from the right, transposed	$dJ = \langle P \odot \nabla_P J, dT \rangle - \langle (P \odot \nabla_P J)\mathbf{1}, (P \odot dT)\mathbf{1} \rangle$
move $\mathbf{1}$ across from the right, transposed	$= \langle P \odot \nabla_P J, dT \rangle - \langle ((P \odot \nabla_P J)\mathbf{1})\mathbf{1}^\mathsf{T}, P \odot dT \rangle$
move P across, no transpose	$= \langle P \odot \nabla_P J, dT \rangle - \langle P \odot ((P \odot \nabla_P J)\mathbf{1})\mathbf{1}^\mathsf{T}, dT \rangle$
linearity in the first argument	$= \left\langle P \odot \left(\nabla_P J - ((P \odot \nabla_P J)\mathbf{1})\mathbf{1}^\mathsf{T}\right), dT \right\rangle$
read off	$\nabla_T J = \boxed{P \odot \left(\nabla_P J - ((P \odot \nabla_P J)\mathbf{1})\mathbf{1}^\mathsf{T}\right)}$

The Long Way: Combining the Two

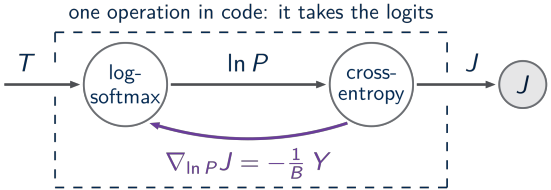
Given $\nabla_P J = -\frac{1}{B}\, Y \oslash P$, find $\nabla_T J$ with the softmax's local gradient.

step 1: $\nabla_P J = -\frac{1}{B}\, Y \oslash P$	$P \odot \nabla_P J = -\frac{1}{B}\, P \odot (Y \oslash P)$
multiplying by P undoes the Hadamard division by P	$= -\frac{1}{B}\, Y$
multiply both sides by $\mathbf{1}$	$(P \odot \nabla_P J)\mathbf{1} = -\frac{1}{B}\, Y\mathbf{1}$
each row of Y has a single 1, so $Y\mathbf{1} = \mathbf{1}$	$= -\frac{1}{B}\, \mathbf{1}$
step 2: the local gradient of the softmax	$\nabla_T J = P \odot \left(\nabla_P J - ((P \odot \nabla_P J)\mathbf{1})\mathbf{1}^\mathsf{T}\right)$
substitute $\nabla_P J$ and $(P \odot \nabla_P J)\mathbf{1}$	$= P \odot \left(-\frac{1}{B}\, Y \oslash P + \frac{1}{B}\, \mathbf{1}\mathbf{1}^\mathsf{T}\right)$
distribute the Hadamard product	$= -\frac{1}{B}\, P \odot (Y \oslash P) + \frac{1}{B}\, P \odot \mathbf{1}\mathbf{1}^\mathsf{T}$
$P \odot (Y \oslash P) = Y$, and $P \odot \mathbf{1}\mathbf{1}^\mathsf{T} = P$	$= -\frac{1}{B}\, Y + \frac{1}{B}\, P$
collect the terms	$\nabla_T J = \boxed{\frac{1}{B}(P - Y)}$

In exact arithmetic the division by P cancels, after 24 lines; in floating point it happens first.

Splitting the Graph at ln P

Combining the softmax and the cross-entropy on the logits is numerically stable.



$\ell = -t_y + m + \ln \sum_c \exp(t_c - m), \quad m = \max_c t_c$

how PyTorch computes it: every exponent is at most 0
and the sum is at least 1, so nothing overflows or divides

true class $y = 1$, logits $\mathbf{t} = (0, 800, 0)$

$\ell = 800 + \ln(e^{-800} + 1 + e^{-800}) = 800$

$m = 800$: a large loss, but finite

$\nabla_{\mathbf{t}} \ell = \mathbf{p} - \mathbf{y} = (-1, 1, 0)$

no division by $\mathbf{p}$

A computation is numerically stable when its rounding errors stay small instead of growing. Subtracting the largest logit keeps every exponential at most 1 and nothing divides by P, while the chain rule and the result are unchanged.

The Two Splits in PyTorch

Softmax then log breaks; log-softmax then NLL, and `F.cross_entropy`, do not.

```
t = torch.tensor([[0., 800., 0.]], requires_grad=True)   # one example's logits
y = torch.tensor([0])                                    # its true class (PyTorch counts from 0)
```

softmax, then log, then NLL

```
# log + NLL: cross-entropy of p
p = torch.softmax(t, dim=1)
# [[0., 1., 0.]]: e^-800 -> 0
logp = torch.log(p)
# [[-inf, 0., -inf]]
loss = F.nll_loss(logp, y)
# inf
loss.backward()
# t.grad: [[nan, nan, nan]]
```

log-softmax, then NLL

```
# softmax and log in one step
logp = F.log_softmax(t, dim=1)
# t - 800: [[-800., 0., -800.]]
# p is never formed

loss = F.nll_loss(logp, y)
# 800.0
loss.backward()
# t.grad: [[-1., 1., 0.]] = p - y
```

F.cross_entropy

```
# log_softmax + nll_loss in one
loss = F.cross_entropy(t, y)
# takes the logits t, not p




# 800.0
loss.backward()
# t.grad: [[-1., 1., 0.]] = p - y
```

The cross-entropy $-\ln p_y$ is a function of $\mathbf{p}$, but F.cross_entropy takes logits, so it contains the softmax.

The Gradient at the Logits

Given the one-hot true labels Y and the loss $J = -\frac{1}{B}\langle Y, \ln P\rangle$, find $\nabla_T J$.

cross-entropy loss of the batch	$J = -\frac{1}{B}\langle Y, \ln P\rangle$
Y is a constant matrix	$dJ = -\frac{1}{B}\langle Y, d\ln P\rangle$
the differential of the log-probabilities	$= -\frac{1}{B}\langle Y,\ dT - \big((P \odot dT)\mathbf{1}\big)\mathbf{1}^\mathsf{T}\rangle$
linearity in the second argument	$= -\frac{1}{B}\big(\langle Y, dT\rangle - \langle Y,\ \big((P \odot dT)\mathbf{1}\big)\boxed{\mathbf{1}^\mathsf{T}}\rangle\big)$
move $\mathbf{1}^\mathsf{T}$ across from the right, transposed	$= -\frac{1}{B}\big(\langle Y, dT\rangle - \langle Y\boxed{\mathbf{1}},\ (P \odot dT)\mathbf{1}\rangle\big)$
each row of Y has a single 1, so $Y\mathbf{1} = \mathbf{1}$	$= -\frac{1}{B}\big(\langle Y, dT\rangle - \langle \mathbf{1},\ (P \odot dT)\mathbf{1}\rangle\big)$
both add up every entry of $P \odot dT$	$= -\frac{1}{B}\big(\langle Y, dT\rangle - \langle P, dT\rangle\big)$
linearity in the first argument	$= \big\langle \frac{1}{B}(P - Y),\ dT\big\rangle$
read off: predictions P minus true labels Y	$\nabla_T J = \boxed{\frac{1}{B}(P - Y)}$

Backward Steps 1 and 2 in Full

Steps 1 and 2 give $\nabla_T J = \frac{1}{B}(P - Y)$, already in the forward arrays.

$\nabla_T J = \frac{1}{B}(P - Y)$

the loss and the softmax differentiated together: nothing is divided by P

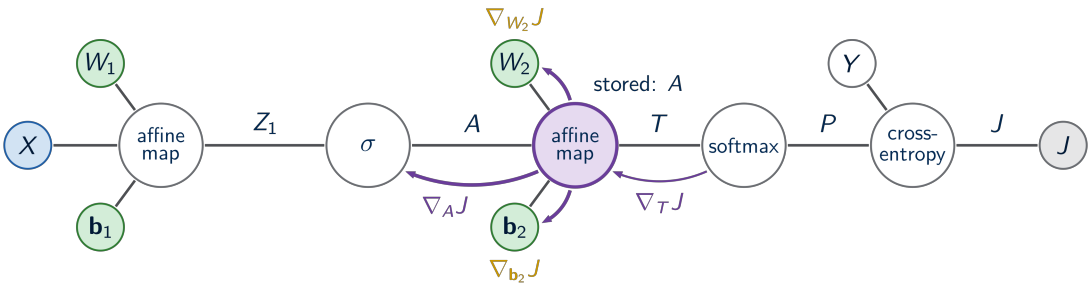
$$\frac{1}{2}\left(\begin{array}{|c|c|c|} \hline 0.610 & 0.143 & 0.247 \\ \hline 0.535 & 0.195 & 0.270 \\ \hline \end{array} - \begin{array}{|c|c|c|} \hline 1 & 0 & 0 \\ \hline 0 & 0 & 1 \\ \hline \end{array} \right) = \begin{array}{|c|c|c|} \hline -0.195 & 0.071 & 0.123 \\ \hline 0.267 & 0.097 & -0.365 \\ \hline \end{array}$$

P Y $\nabla_T J$

the prediction minus the truth, divided by B

Backward Step 3: The Second Affine Map

The second affine map receives $\nabla_T J$ and sends back gradients to A, W_2 and $\mathbf{b}_2$.



Its local rule uses the A it stored going forward and the parameter W_2. The two gold gradients are results, and $\nabla_A J$ continues back to the sigmoid.

The Differential of the Second Affine Map

Given $\nabla_T J$ and $T = AW_2 + \mathbf{1}\mathbf{b}_2^\mathsf{T}$, write dJ as a sum of inner products, one for each of dA, dW_2 and $d\mathbf{b}_2$.

definition of $\nabla_T J$	$dJ = \langle \nabla_T J,\ dT \rangle$
$T = AW_2 + \mathbf{1}\mathbf{b}_2^\mathsf{T}$	$= \langle \nabla_T J,\ d(AW_2 + \mathbf{1}\mathbf{b}_2^\mathsf{T}) \rangle$
sum rule	$= \langle \nabla_T J,\ d(AW_2) + d(\mathbf{1}\mathbf{b}_2^\mathsf{T}) \rangle$
product rule	$= \langle \nabla_T J,\ (dA)\, W_2 + A\, dW_2 + d(\mathbf{1}\mathbf{b}_2^\mathsf{T}) \rangle$
constant factor rule: $\mathbf{1}$ is a constant matrix	$= \langle \nabla_T J,\ (dA)\, W_2 + A\, dW_2 + \mathbf{1}\, d\mathbf{b}_2^\mathsf{T} \rangle$
linearity in the second argument	$= \boxed{\langle \nabla_T J, (dA)\, W_2 \rangle + \langle \nabla_T J, A\, dW_2 \rangle + \langle \nabla_T J, \mathbf{1}\, d\mathbf{b}_2^\mathsf{T} \rangle}$

Gradients of the Second Affine Map

Given $dJ = \langle \nabla_T J, (dA) W_2 \rangle + \langle \nabla_T J, A\, dW_2 \rangle + \langle \nabla_T J, \mathbf{1}\, d\mathbf{b}_2^\mathsf{T} \rangle$, find $\nabla_A J$, $\nabla_{W_2} J$ and $\nabla_{\mathbf{b}_2} J$.

given	$dJ = \langle \nabla_T J, (dA)\, W_2 \rangle + \langle \nabla_T J, A\, dW_2 \rangle + \langle \nabla_T J, \mathbf{1}\, d\mathbf{b}_2^\mathsf{T} \rangle$
move W_2 across from the right, transposed	$= \langle (\nabla_T J)\, W_2^\mathsf{T}, dA \rangle + \langle \nabla_T J, A\, dW_2 \rangle + \langle \nabla_T J, \mathbf{1}\, d\mathbf{b}_2^\mathsf{T} \rangle$
move A across from the left, transposed	$= \langle (\nabla_T J) W_2^\mathsf{T}, dA \rangle + \langle A^\mathsf{T} \nabla_T J, dW_2 \rangle + \langle \nabla_T J, \mathbf{1}\, d\mathbf{b}_2^\mathsf{T} \rangle$
move $\mathbf{1}$ across from the left, transposed	$= \langle (\nabla_T J) W_2^\mathsf{T}, dA \rangle + \langle A^\mathsf{T} \nabla_T J, dW_2 \rangle + \langle \mathbf{1}^\mathsf{T} \nabla_T J, d\mathbf{b}_2^\mathsf{T} \rangle$
transpose both arguments of the last term	$= \langle (\nabla_T J) W_2^\mathsf{T}, dA \rangle + \langle A^\mathsf{T} \nabla_T J, dW_2 \rangle + \langle (\nabla_T J)^\mathsf{T} \mathbf{1}, d\mathbf{b}_2 \rangle$
read off each	$\nabla_A J = (\nabla_T J) W_2^\mathsf{T}, \quad \nabla_{W_2} J = A^\mathsf{T} \nabla_T J, \quad \nabla_{\mathbf{b}_2} J = (\nabla_T J)^\mathsf{T} \mathbf{1}$

Backward Step 3 in Full

Substituting $\nabla_T J$ writes step 3's three gradients in the forward arrays.

$$\nabla_A J = (\nabla_T J) W_2^\mathsf{T} = \boxed{\tfrac{1}{B}(P - Y)} W_2^\mathsf{T}$$

$$\nabla_{W_2} J = A^\mathsf{T} \nabla_T J = A^\mathsf{T} \boxed{\tfrac{1}{B}(P - Y)}$$

$$\nabla_{\mathbf{b}_2} J = (\nabla_T J)^\mathsf{T} \mathbf{1} = \left(\boxed{\tfrac{1}{B}(P - Y)}\right)^\mathsf{T} \mathbf{1}$$

$\nabla_A J$

−0.266	−0.052	−0.071
0.170	0.462	−0.097

$\nabla_{W_2} J$

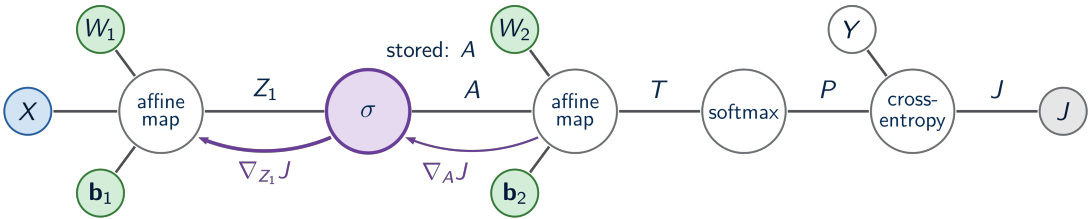
0.010	0.139	−0.149
0.069	0.161	−0.230
0.036	0.084	−0.121

$\nabla_{\mathbf{b}_2} J$

0.073
0.169
−0.242

Backward Step 4: The Sigmoid

The sigmoid node receives $\nabla_A J$ and sends back the gradient with respect to Z_1.



Its local rule uses the A it stored going forward, because the derivative of the sigmoid is written with its own output.

Gradients Through the Sigmoid

Given $\nabla_A J$ and $A = \sigma(Z_1)$, find $\nabla_{Z_1} J$.

definition of the gradient $\nabla_A J$	$dJ = \langle \nabla_A J,\ dA \rangle$
$A = \sigma(Z_1)$	$= \langle \nabla_A J,\ d\sigma(Z_1) \rangle$
element-wise rule	$= \langle \nabla_A J,\ \sigma'(Z_1) \odot dZ_1 \rangle$
$\sigma' = \sigma(1-\sigma)$ on each entry	$= \langle \nabla_A J,\ \sigma(Z_1) \odot \big(1 - \sigma(Z_1)\big) \odot dZ_1 \rangle$
$\sigma(Z_1) = A$	$= \langle \nabla_A J,\ A \odot (1 - A) \odot dZ_1 \rangle$
move the Hadamard factor across, with no transpose	$= \langle A \odot (1 - A) \odot \nabla_A J,\ dZ_1 \rangle$
the Hadamard product commutes	$= \langle \nabla_A J \odot A \odot (1 - A),\ dZ_1 \rangle$
read off	$\nabla_{Z_1} J = \nabla_A J \odot A \odot (1 - A)$

Backward Step 4 in Full

Substituting $\nabla_A J$ writes step 4's gradient in the forward arrays.

$$\nabla_{Z_1} J = \nabla_A J \odot A \odot (1 - A) = \left(\boxed{\tfrac{1}{B}(P - Y) W_2^{\mathsf{T}}} \right) \odot A \odot (1 - A)$$

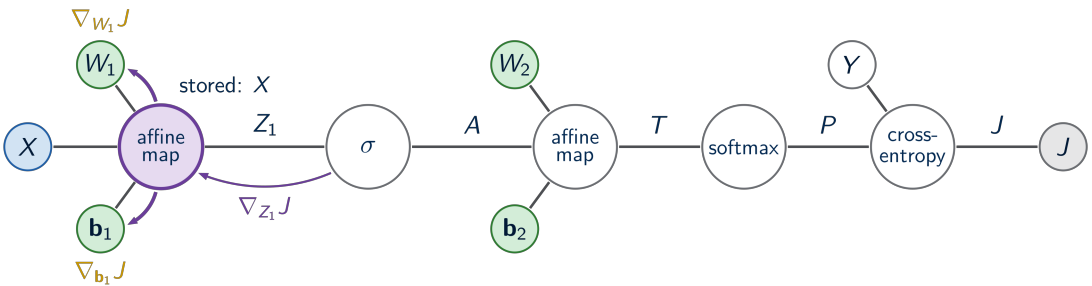
step 3's $\nabla_A J$ substituted

$\nabla_A J$				$A \odot (1-A)$				$\nabla_{Z_1} J$		
−0.266	−0.052	−0.071	$\odot$	0.045	0.045	0.250	$=$	−0.012	−0.002	−0.018
0.170	0.462	−0.097		0.197	0.045	0.250		0.033	0.021	−0.024

saturated entries of A, near 0 or 1, shrink the gradient that passes through them

Backward Step 5: The First Affine Map

The first affine map receives $\nabla_{Z_1} J$ and sends back the gradients for W_1 and $\mathbf{b}_1$.



Its local rule uses the X it stored going forward. X is the data, a constant matrix, so no gradient is sent to it.

Gradients of the First Affine Map

Given $\nabla_{Z_1} J$ and $Z_1 = XW_1 + \mathbf{1}\mathbf{b}_1^\mathsf{T}$, find $\nabla_{W_1} J$ and $\nabla_{\mathbf{b}_1} J$.

definition of the gradient $\nabla_{Z_1} J$	$dJ = \langle \nabla_{Z_1} J,\ dZ_1 \rangle$
$Z_1 = XW_1 + \mathbf{1}\mathbf{b}_1^\mathsf{T}$, then the sum rule	$= \langle \nabla_{Z_1} J,\ d(XW_1) + d(\mathbf{1}\mathbf{b}_1^\mathsf{T}) \rangle$
constant factor rule: X and $\mathbf{1}$ are constant matrices	$= \langle \nabla_{Z_1} J,\ X\,dW_1 + \mathbf{1}\,d\mathbf{b}_1^\mathsf{T} \rangle$
linearity in the second argument	$= \langle \nabla_{Z_1} J,\ X\,dW_1 \rangle + \langle \nabla_{Z_1} J,\ \mathbf{1}\,d\mathbf{b}_1^\mathsf{T} \rangle$
move X across from the left, transposed	$= \langle X^\mathsf{T} \nabla_{Z_1} J,\ dW_1 \rangle + \langle \nabla_{Z_1} J,\ \mathbf{1}\,d\mathbf{b}_1^\mathsf{T} \rangle$
move $\mathbf{1}$ across from the left, transposed	$= \langle X^\mathsf{T} \nabla_{Z_1} J,\ dW_1 \rangle + \langle \mathbf{1}^\mathsf{T} \nabla_{Z_1} J,\ d\mathbf{b}_1^\mathsf{T} \rangle$
transpose both arguments of the second term	$= \langle X^\mathsf{T} \nabla_{Z_1} J,\ dW_1 \rangle + \langle (\nabla_{Z_1} J)^\mathsf{T} \mathbf{1},\ d\mathbf{b}_1 \rangle$
read off each	$\nabla_{W_1} J = \boxed{X^\mathsf{T} \nabla_{Z_1} J}$ $\quad \nabla_{\mathbf{b}_1} J = \boxed{(\nabla_{Z_1} J)^\mathsf{T} \mathbf{1}}$

Backward Step 5 in Full

Substituting $\nabla_{Z_1} J$ writes the first layer's gradients in the forward arrays.

$$\nabla_{W_1} J = X^\top \nabla_{Z_1} J = X^\top \left[\left(\tfrac{1}{B}(P - Y) W_2^\top \right) \odot A \odot (1 - A) \right]$$

$$\nabla_{\mathbf{b}_1} J = (\nabla_{Z_1} J)^\top \mathbf{1} = \left[\left(\tfrac{1}{B}(P - Y) W_2^\top \right) \odot A \odot (1 - A) \right]^\top \mathbf{1}$$

$\nabla_{W_1} J$

−0.012	−0.002	−0.018
0.033	0.021	−0.024
0.009	0.016	−0.060

$\nabla_{\mathbf{b}_1} J$

0.021
0.019
−0.042

Putting It Together: Passing Gradients Back

Each gradient passed back contains the one before it.

step	local rule	in full
①②	$\nabla_T J = \frac{1}{B}(P - Y)$	$= \frac{1}{B}(P - Y)$
③	$\nabla_A J = (\nabla_T J)\, W_2^{\mathsf{T}}$	$= \underbrace{\frac{1}{B}(P - Y)}_{\nabla_T J}\, W_2^{\mathsf{T}}$
④	$\nabla_{Z_1} J = \nabla_A J \odot A \odot (1 - A)$	$= \underbrace{\underbrace{\left(\frac{1}{B}(P - Y)\, W_2^{\mathsf{T}}\right)}_{\nabla_A J} \odot A \odot (1 - A)}_{\nabla_{Z_1} J}$

Each gradient passed back has its own colour: $\nabla_T J$ purple, $\nabla_A J$ magenta, $\nabla_{Z_1} J$ teal. In a full form, each piece keeps the colour of the gradient it came from, and the new local factor keeps its own colour.

Putting It Together: The Parameter Gradients

Each parameter's gradient is a local factor times its layer's gradient.

step	local rule	in full
③	$\boxed{\nabla_{W_2} J} = A^\top \nabla_T J$	$= A^\top \underbrace{\tfrac{1}{B}(P - Y)}_{\nabla_T J}$
	$\boxed{\nabla_{\mathbf{b}_2} J} = (\nabla_T J)^\top \mathbf{1}$	$= \big(\underbrace{\tfrac{1}{B}(P - Y)}_{\nabla_T J}\big)^\top \mathbf{1}$
⑤	$\boxed{\nabla_{W_1} J} = X^\top \nabla_{Z_1} J$	$= X^\top \big[\underbrace{\big(\underbrace{\tfrac{1}{B}(P - Y)}_{\nabla_T J}\, W_2^\top\big)}_{\nabla_A J} \odot A \odot (1 - A)\big]$ (with $\nabla_{Z_1} J$ over the whole bracket)
	$\boxed{\nabla_{\mathbf{b}_1} J} = (\nabla_{Z_1} J)^\top \mathbf{1}$	$= \big[\underbrace{\big(\tfrac{1}{B}(P - Y)\, W_2^\top\big) \odot A \odot (1 - A)\big]}_{\nabla_{Z_1} J}^\top \mathbf{1}$

The Backward Pass in PyTorch

Written by hand, the backward pass gives the same gradients as PyTorch's autograd.

by hand: one line per local rule

```
# X, W1, b1, W2, b2, Y: running example; B = 2
# forward, storing X, A and P
A = torch.sigmoid(X @ W1 + b1)
P = torch.softmax(A @ W2 + b2, dim=1)
# backward, from the loss to the parameters
grad_T  = (P - Y) / B          # steps 1, 2
grad_A  = grad_T @ W2.T        # step 3
grad_W2 = A.T @ grad_T
grad_b2 = grad_T.sum(dim=0)
grad_Z1 = grad_A * A * (1 - A) # step 4
grad_W1 = X.T @ grad_Z1        # step 5
grad_b1 = grad_Z1.sum(dim=0)
```

autograd: `loss.backward()`

```
# the same network, differentiated by PyTorch
y = Y.argmax(dim=1)            # tensor([0, 2])
for p in (W1, b1, W2, b2):
    p.requires_grad_()         # record
T = torch.sigmoid(X @ W1 + b1) @ W2 + b2
loss = F.cross_entropy(T, y)   # 0.9014
loss.backward()                # fills p.grad

torch.allclose(W1.grad, grad_W1)  # True
torch.allclose(b1.grad, grad_b1)  # True
torch.allclose(W2.grad, grad_W2)  # True
torch.allclose(b2.grad, grad_b2)  # True
```

L2 Regularization

Given $J_\lambda = J + \lambda\Big(\|W_1\|^2 + \|W_2\|^2\Big)$, find the gradients of J_λ.

given, with $\|W\|^2 = \langle W, W\rangle$
$$J_\lambda = J + \lambda\big(\langle W_1, W_1\rangle + \langle W_2, W_2\rangle\big)$$

sum rule
$$dJ_\lambda = dJ + \lambda\big(d\langle W_1, W_1\rangle + d\langle W_2, W_2\rangle\big)$$

product rule and symmetry
$$= dJ + 2\lambda\langle W_1, dW_1\rangle + 2\lambda\langle W_2, dW_2\rangle$$

write out dJ
$$= \langle \nabla_{W_1} J, dW_1\rangle + \langle \nabla_{\mathbf{b}_1} J, d\mathbf{b}_1\rangle + \langle \nabla_{W_2} J, dW_2\rangle + \langle \nabla_{\mathbf{b}_2} J, d\mathbf{b}_2\rangle$$
$$+ 2\lambda\langle W_1, dW_1\rangle + 2\lambda\langle W_2, dW_2\rangle$$

linearity in the first argument
$$= \langle \nabla_{W_1} J + 2\lambda W_1,\ dW_1\rangle + \langle \nabla_{W_2} J + 2\lambda W_2,\ dW_2\rangle$$
$$+ \langle \nabla_{\mathbf{b}_1} J, d\mathbf{b}_1\rangle + \langle \nabla_{\mathbf{b}_2} J, d\mathbf{b}_2\rangle$$

read off
$$\nabla_{W_1} J_\lambda = \boxed{\nabla_{W_1} J + 2\lambda W_1}, \qquad \nabla_{W_2} J_\lambda = \boxed{\nabla_{W_2} J + 2\lambda W_2}$$

biases are not penalized
$$\nabla_{\mathbf{b}_1} J_\lambda = \nabla_{\mathbf{b}_1} J, \qquad \nabla_{\mathbf{b}_2} J_\lambda = \nabla_{\mathbf{b}_2} J$$

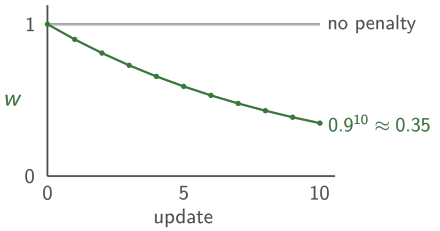
in full
$$\nabla_{W_1} J_\lambda = X^\mathsf{T}\big[\big(\tfrac{1}{B}(P - Y)W_2^\mathsf{T}\big) \odot A \odot (1 - A)\big] + 2\lambda W_1$$
$$\nabla_{W_2} J_\lambda = A^\mathsf{T}\,\tfrac{1}{B}(P - Y) + 2\lambda W_2$$

Weight Decay

Given the learning rate η and $\nabla_W J_\lambda = \nabla_W J + 2\lambda W$, find the gradient descent update of each weight matrix W.

gradient descent update	$W \leftarrow W - \eta \nabla_W J_\lambda$
$\nabla_W J_\lambda = \nabla_W J + 2\lambda W$	$= W - \eta (\nabla_W J + 2\lambda W)$
collect the terms in W	$= (1 - 2\eta\lambda) W - \eta \nabla_W J$

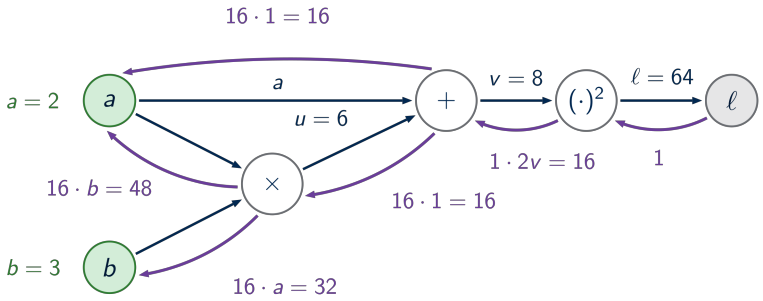
with $\eta = 0.5$ and $\lambda = 0.1$, each update multiplies w by $1 - 2\eta\lambda = 0.9$



Each update first multiplies every weight by $1 - 2\eta\lambda$, a number slightly below 1, then takes the usual step against $\nabla_W J$. With no gradient from J, the weights shrink by the same fraction every update: that is the decay.

Reverse-Mode Automatic Differentiation

Reverse-mode automatic differentiation is the backward pass, run by the computer on any graph.



one backward sweep gives both

$\partial \ell / \partial a = \boxed{48 + 16 = 64}$

a has two paths: their gradients add

$\partial \ell / \partial b = \boxed{32}$

check: $\ell = (ab + a)^2$

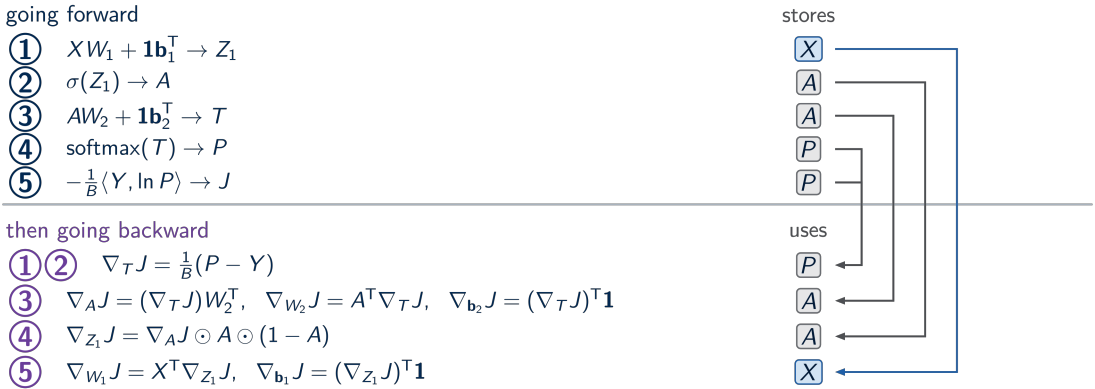
$\partial \ell / \partial a = 2(ab + a)(b + 1) = 64$

$\partial \ell / \partial b = 2(ab + a)\, a = 32$

Going forward, each node computes its value. Going backward from 1 at ℓ, each arc multiplies by its node's local derivative, and where an input feeds two nodes, its gradients add.

The Forward and Backward Passes

The forward pass stores what the backward pass uses, in reverse order.



Each line on the right joins a stored array to the backward step that uses it.